20、Dcm\Dem $19服务 DTC-$14清除DTC-$85控制DTC

卖家'漩涡遨游aa' 专属资料，非在漩涡遨游aa 购买资料，不提供答疑(一次购买仅提供有限次数10次答疑)

https://blog.csdn.net/fly_xiaocaibi88/category_11979307.html

以下是UDS（统一诊断服务）中0x19服务的核心子功能示例及解析，结合ISO 14229标准与实际应用场景整理而成：

1、基本概念

DTC(Diagnostic Trouble Code)即诊断故障码，DTC是汽车电子系统故障的"身份ID",当ECU检测到某个系统或部件出现故障时，它会生成一个DTC，每个DTC都唯一对应一个特定的故障类型或诊断事件。通过读取和分析DTC，技术人员可以了解故障的具体信息，如触发条件、解除条件以及系统的功能表现等。

2、DTC的构成

根据ISO 15301-6和ISO 14229-1的故障诊断码格式规定，故障码信息由四字节组成,如下表

DTC 信息格式			
DTCHighByte(Hex)	DTCMiddleByte(Hex)	DTCLowByte(Hex)	StatusOfDTC

其中前三个字节表示故障诊断码即DTC，StatusOfDTC表示故障码状态。其中DTCHighByte、DTCMiddleByte这两个字节表示故障内码，对应五位标准故障码，第一位是字母，后面四位是数字；而故障代码的低字节代表故障失效类型信息。

故障内码	DTCHighByte(Hex)								DTCMiddleByte(Hex)							
	Bit 15	Bit 14	Bit 13	Bit 12	bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	bit 0
五位标准故障码	第一位		第二位		第三位				第四、五位							

3、故障码解析

1、故障诊断码高字节的第一位是字母，表示故障所属系统

故障内码bit14bit15	标准故障码表示字符	所属系统
0x00	P	Powertrain动力系统故障
0x01	C	Chassis底盘故障
0x10	B	Body车身故障
0x11	U	Network网络故障

2、故障诊断码高字节的第二位是数字0、1、2或3，表示故障类型

故障内码bit13bit12	标准故障码表示字符	故障类型
0x00	0	ISOSAE标准定义故障码
0x01	1	制造商自定义故障码
0x10	2	ISOSAE保留码
0x11	3	ISOSAE保留码

3、故障诊断码高字节的第三位是数字，表示故障所属的子系统，以动力系统故障为例

0：表示燃油和空气计量辅助排放控制整个系统；
1：表示燃油和空气计量系统；
2：表示燃油和空气计量系统（喷油器）
3：表示点火系统；
4：表示废气控制系统；
5：表示巡航、怠速控制系统；
6：车载电脑和输出信号；
7、8：传动系统控制；

4、标准故障码的第四、五位也是数字，表示具体故障对象和类型;DTCLowByte则是描述故障种类和子类型，该部分内容遵循ISO 15031-6,对于不需要该字节的DTC,可填充为0x00。

具体故障种类在ISO 15031-6中详细定义：

Table D.2 — DTC Failure Category definition

DTCFailureTypeByte Category Definitions		
High Nibble (0000b - 1111b)	**Category # (hex)**	**Category Description**
0000	0	**General Failure Information** 一般故障信息 This category includes all other categories and is used when the fault within that failure category is unique (not amenable to standardization through assignment of a new Sub Type) or when the detected fault is best described by two or more Sub Types within that Failure Category.
0001	1	**General Electrical Failures** 一般电气故障 This category includes standard wiring failure modes (i.e., shorts and opens), and direct current (DC) quantities related by Ohm's Law.
0010	2	**General Signal Failures** 一般信号故障 This category includes quantities related to amplitude, frequency or rate of change, and wave shape.
0011	3	**FM (Frequency Modulated) / PWM (Pulse Width Modulated) Failures** FM(调频)/ PWM(脉宽调制)故障 This category includes faults related to Frequency Modulated (FM) and Pulse Width Modulated (PWM) inputs and outputs of the server. This category also includes faults where position is determined by counts.
0100	4	**System Internal Failures** 系统内部故障 This category includes faults related to memory, software, and internal electrical circuitry; requiring component (server, sensor, etc.) replacement.
0101	5	**System Programming Failures** 系统编程故障 This category includes faults related to operational software, calibrations, and options; remedied by configuring/programming a part of the system (server, sensor, etc.).
0110	6	**Algorithm Based Failures** 基于算法的故障 This category includes faults based on comparing two or more input parameters for plausibility or comparing a single parameter to itself with respect to time.
0111	7	**Mechanical Failures** 机械故障 This category includes faults detected by inappropriate motion in response to server related input/controlled output.
1000	8	**Bus Signal / Message Failures** 总线信号/消息故障 This category includes faults related to bus hardware and signal integrity. This category is also used when the physical input for a signal is located in one server and another server diagnoses the circuit or inhibits operation due to a reported failure of that circuit.
1001	9	**Component Failures** 部件故障 This category includes faults related to component failures (including parametric, performance assembly and operating environment failures).
1010 – 1110	A - E	**ISO/SAE reserved** This range of values is reserved by the document for future expansion.

⚙️ 一、19 01：读取匹配状态掩码的DTC数量

- **功能**：统计符合指定状态条件（如当前/历史故障）的DTC数量。
- **请求示例**：`19 01 [状态掩码]`
  - `19 01 01`：请求当前故障（TestFailed）的DTC数量
  - `19 01 09`：请求当前故障或已确认故障（ConfirmedDTC）的DTC数量
- **响应示例**：`59 01 [支持掩码] [DTC格式] [数量高字节] [数量低字节]`
  - `59 01 09 01 00 03`：支持掩码为0x09，符合条件DTC共3个

UDS🔍服务19 01按状态掩码读DTC数量

19 01服务

Message direction	client -> server	
Message type	Request	
A_Data byte	Description	Byte value
#1	ReadDTCInformation Request SID	19
#2	SubFunction = 按状态掩码报告DTC数量	01
#3	DTCByStatusMask	09

Message direction	server -> client		
Message type	Response		
A_Data byte	Description	Byte value	
#1	ReadDTCInformation Response SID	59	
#2	SubFunction	01	
#3	DtcAvailibaleMask	09	
#4	DTCFormatIdentifiers	00	ISO15031 DTCFormat
#5	DTCCountHighByte	00	
#6	DTCCountLowByte	08	ey:08 代表DTC数量

🔍 二、19 02：读取匹配状态掩码的DTC列表及状态

- **功能**：列出所有符合状态条件的DTC及其实时状态字节。
- **请求示例**：`19 02 [状态掩码]`
  - `19 02 09`：请求状态包含TestFailed或ConfirmedDTC的DTC列表
- **响应示例**：`59 02 [支持掩码] [DTC1] [状态1] [DTC2] [状态2]...`
  - `59 02 FF 81 00 01 2C 81 00 02 2F`：
    - DTC `810001` 状态 `0x2C`（0010 1100）：bit3=1（ConfirmedDTC），bit0=0（非当前故障 → **历史故障**）
    - DTC `810002` 状态 `0x2F`（0010 1111）：bit3=1且bit0=1 → **当前故障**

📸 三、19 04：读取DTC快照（冻结帧）

- **功能**：获取故障触发时刻的实时数据（如电压、车速等）。
- **请求示例**：`19 04 [DTC] [快照记录号]`
  - `19 04 16 A3 00 01`：请求DTC `16A300` 的1号快照数据
- **响应示例**：`59 04 [DTC] [状态] [记录号] [数据组]...`
  - `59 04 16 A3 00 AF 01 10 01 2C 03 FF`：
    - 数据组示例：`0x012C`（DID编号）值为 `0x03FF`（实际数据）
    - 快照记录号：`0x01`

📊 四、19 06：读取DTC扩展数据（如发生次数）

- **功能**：获取故障发生次数（OCC）、老化计数器等附加信息。
- **请求示例**：`19 06 [DTC]`

- `19 06 16 A3 00`：请求DTC `16A300` 的扩展数据

- **响应示例**：`59 06 [DTC] [状态] [OCC] [老化值]...`
  - `59 06 16 A3 00 AF 01 00`：发生次数（OCC）=1，老化计数器=0

📋 五、19 0A：读取所有支持的DTC及状态

- **功能**：返回ECU内部存储的所有DTC（无论是否激活）及其状态。
- **请求示例**：`19 0A`
- **响应示例**：`59 0A [ECU支持的状态掩码] [DTC1] [状态1] [DTC2] [状态2]...`
  - `59 0A FF 12 34 56 24 23 45 05 00 AB CD 01 2F`：
    - ECU支持的状态掩码FF，DTC `123456` 状态 `0x24`，`234505` 状态 `0x00`，`ABCD01` 状态 `0x2F`

🛠️ 六、DTC状态掩码应用示例

掩码值	目标状态	应用场景
`0x01`	TestFailed（当前故障）	实时故障检测
`0x08`	ConfirmedDTC（已确认故障）	历史故障查询
`0x09`	TestFailed | ConfirmedDTC	综合故障筛查

🔬 DTC状态位解析表

Bit位	名称	说明
bit0	TestFailed	当前测试失败（=1为活跃故障）
bit3	ConfirmedDTC	ECU已确认故障（=1为历史故障）
bit6	TestNotCompleted	测试未完成
bit7	WarningIndicator	警告灯激活状态

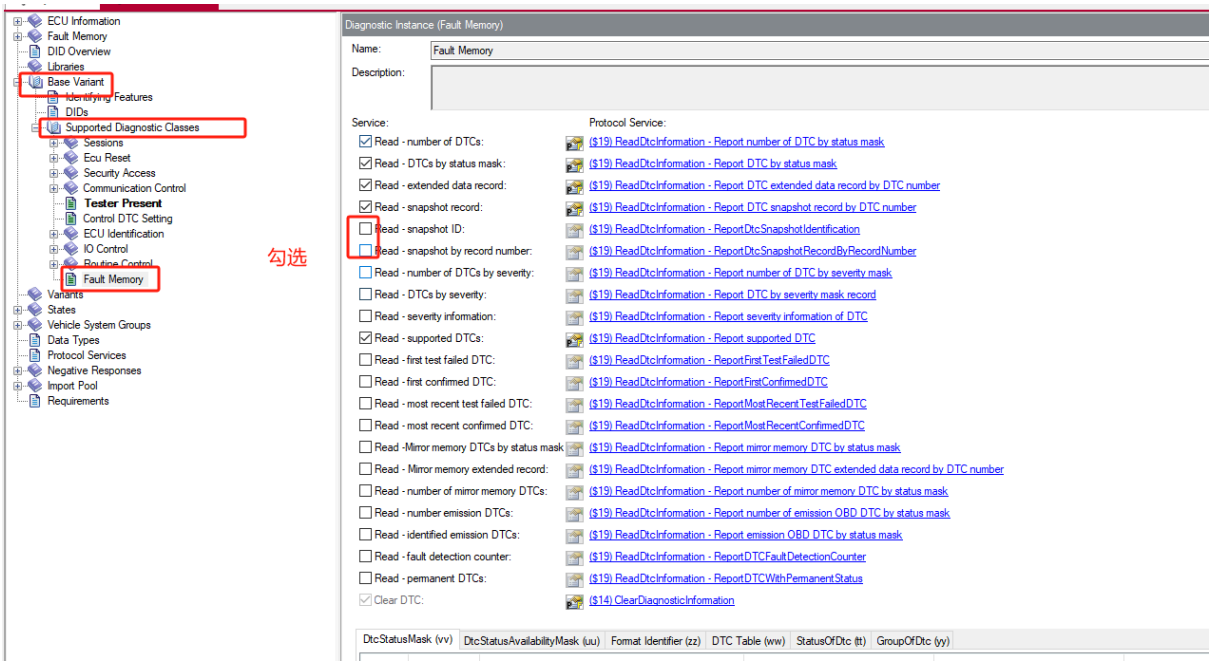
bit	含义	置位条件
bit0	testFailed	置0：testpassed（同个OC内可恢复），0x14服务ClearDTC，Dem_ResetEventStatus调用成功 置1：testfailed, 故障上报
bit1	testFailedThisOperationCycle	置0：OC重新启动，0x14服务ClearDTC 置1：testfailed, 故障上报
bit2	pendingDTC	置0：在整个OC没有testfailed在OC结束时置0，0x14服务ClearDTC，eventmem entry被新故障替换掉 置1：当前或上一个周期内发生过testfailed
bit3	confirmedDTC	置0：达到老化阈值，0x14服务ClearDTC，eventmem entry被新故障替换掉 置1：故障被存储在event memory中且该故障在一定时间（OC）发生了指定的次数
bit4	testNotCompletedSinceLastClear	置0：自上次清除诊断信息以来，DTC测试至少返回一次测试结果（无论testpassed或testfailed） 置1：自上次清除诊断信息后DTC测试尚未运行到完成，0x14服务ClearDTC
bit5	testFailedSinceLastClear	置0：自上次清除诊断信息后DTC测试未显示失败结果；如果满足老化阈值或发生故障记忆溢出，则车辆制造商应负责将该位重置为0 置1：自上次清除诊断信息以来DTC测试至少返回一次失败结果
bit6	testNotCompletedThisOperationCycle	置0：表示在当前OC周期内tested完成 置1：表示在当前OC周期内tested未完成（或OC重启开始），0x14服务ClearDTC
bit7	warningIndicatorRequested	置0：如果没有和DTC相关的警告存在，该位应置0，0x14服务ClearDTC，healing发生 置1：testfailed, 故障上报并达到指定的阈值

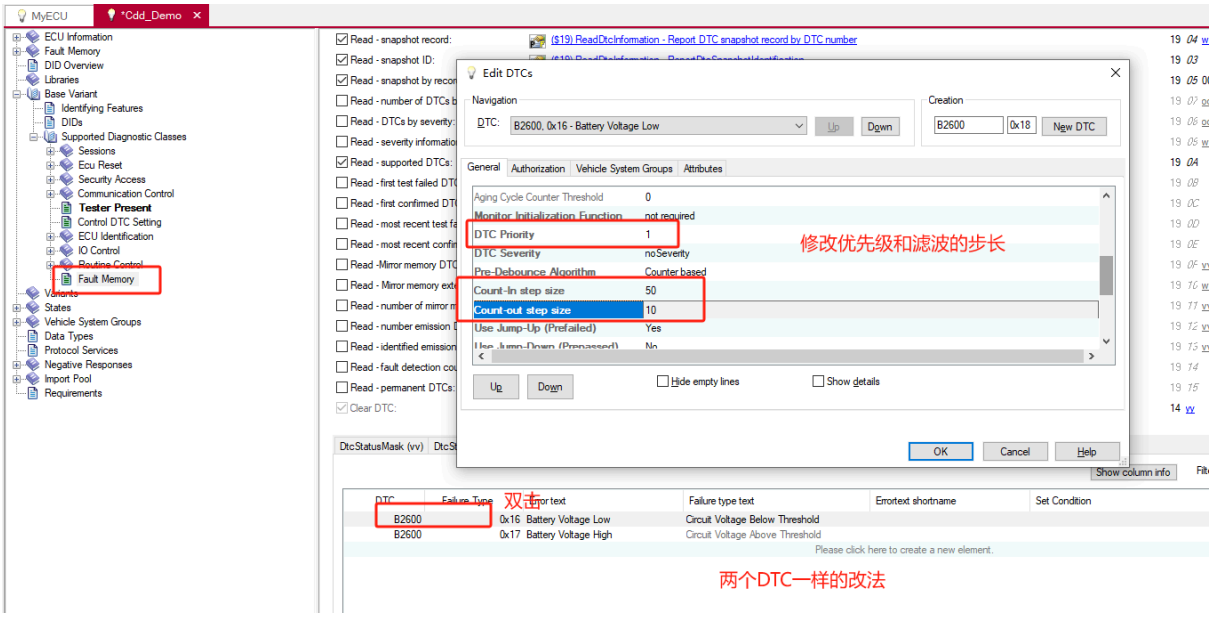
> 💎 关键总结：
>
> - **故障筛选**：通过状态掩码（如 `0x09`）精准定位当前/历史故障。
> - **深度分析**：快照（19 04）记录故障瞬间环境数据，扩展数据（19 06）统计故障频次。
> - **全面诊断**：19 0A可导出ECU内所有DTC支持列表，用于出厂检测或深度排查。实际应用需结合具体DTC定义（如 `P0172` 燃油系统故障）及车辆制造商规范。

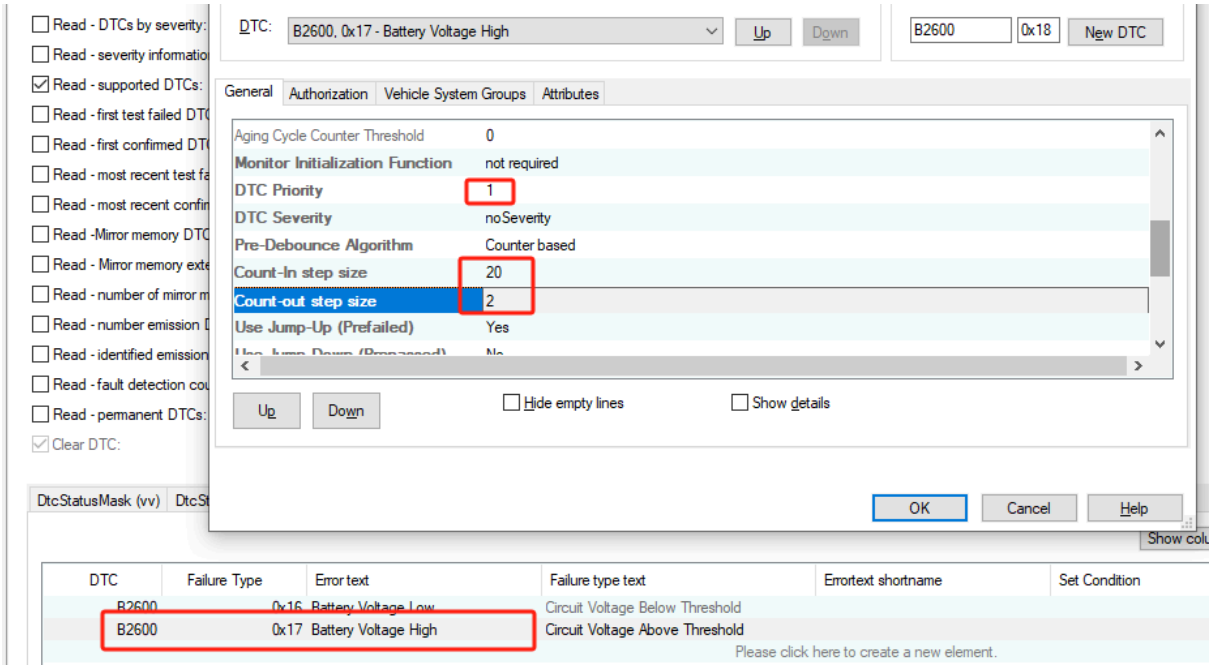
七、Autosar实战配置

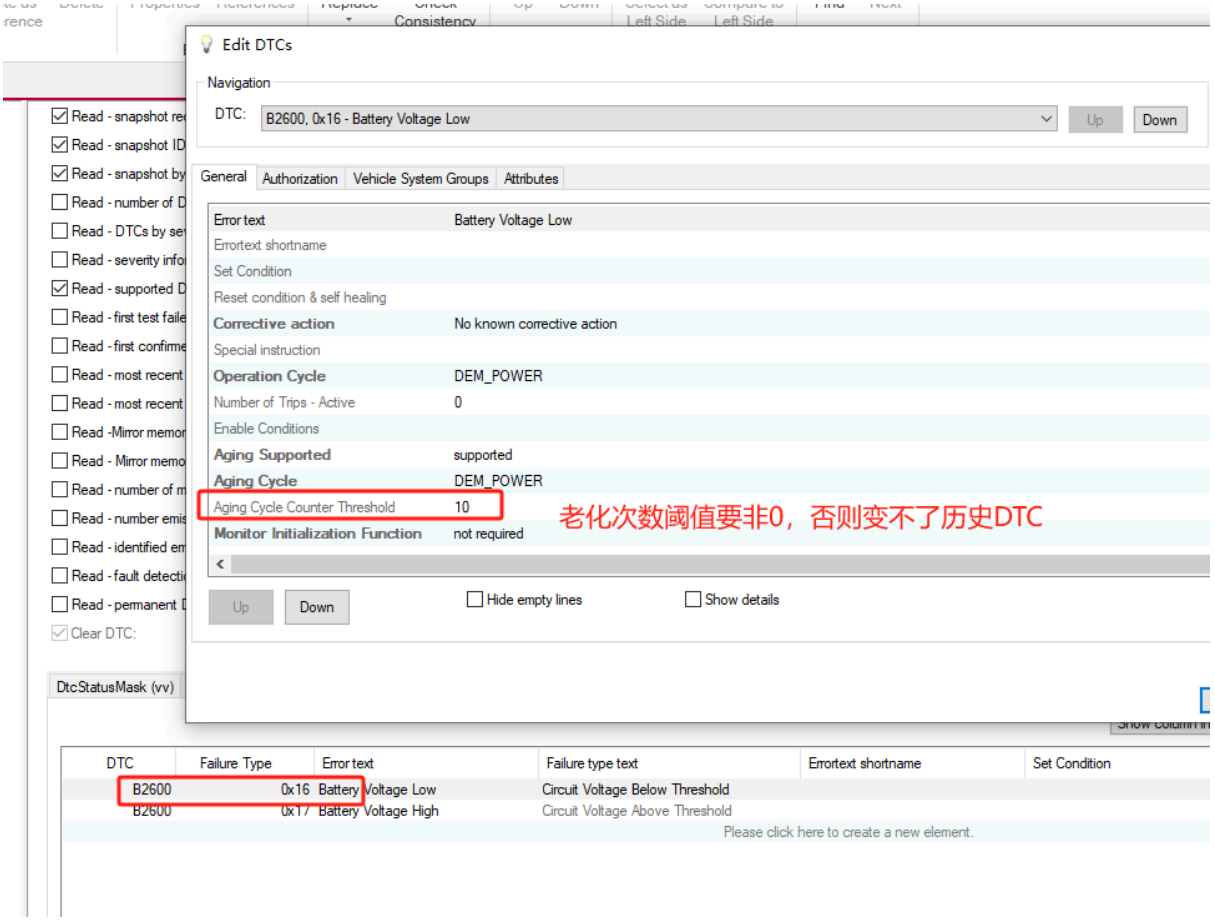
7.1、 $19 01/02/0A 验证

7.1.1、修改cdd（不是必要步骤，有的人可能没有vector的软件）

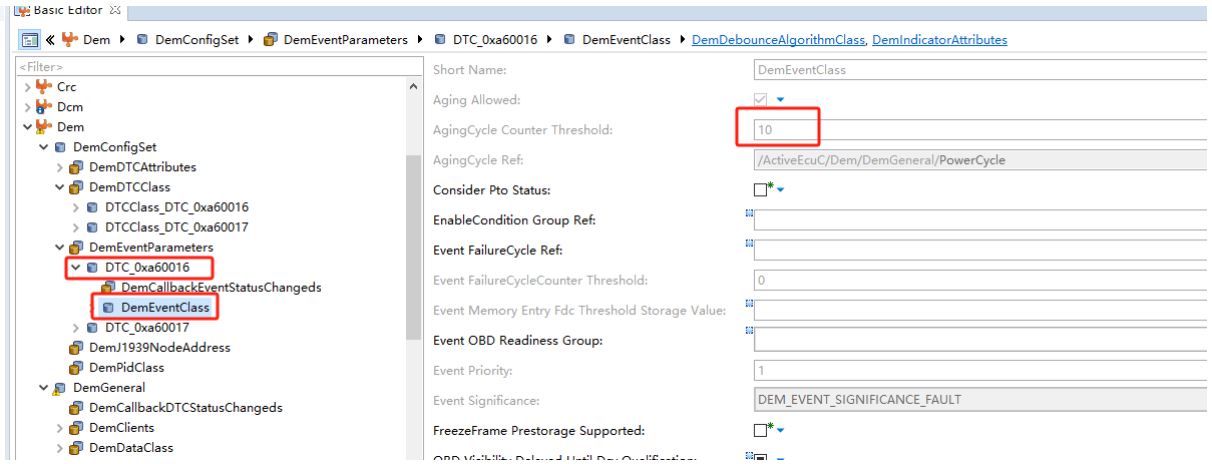

ECU Information
Fault Memory
DID Overview
Libraries
Base Variant
Identifying Features
DIDs
Supported Diagnostic Classes
Sessions
Ecu Reset
Security Access
Communication Control
Tester Present
Control DTC Setting
ECU Identification
IO Control
Routine Control
Fault Memory
Variants
States
Vehicle System Groups
Data Types
Protocol Services
Negative Responses
Import Pool
Requirements
勾选
Diagnostic Instance (Fault Memory)
Name: Fault Memory
Description:
Service:
Protocol Service:
Read - number of DTCs: ($19) ReadDtcInformation - Report number of DTC by status mask
Read - DTCs by status mask: ($19) ReadDtcInformation - Report DTC by status mask
Read - extended data record: ($19) ReadDtcInformation - Report DTC extended data record by DTC number
Read - snapshot record: ($19) ReadDtcInformation - Report DTC snapshot record by DTC number
Read - snapshot ID: ($19) ReadDtcInformation - ReportDtcSnapshotIdentification
Read - snapshot by record number: ($19) ReadDtcInformation - ReportDtcSnapshotRecordByRecordNumber
Read - number of DTCs by severity: ($19) ReadDtcInformation - Report number of DTC by severity mask
Read - DTCs by severity: ($19) ReadDtcInformation - Report DTC by severity mask record
Read - severity information: ($19) ReadDtcInformation - Report severity information of DTC
Read - supported DTCs: ($19) ReadDtcInformation - Report supported DTC
Read - first test failed DTC: ($19) ReadDtcInformation - ReportFirstTestFailedDTC
Read - first confirmed DTC: ($19) ReadDtcInformation - ReportFirstConfirmedDTC
Read - most recent test failed DTC: ($19) ReadDtcInformation - ReportMostRecentTestFailedDTC
Read - most recent confirmed DTC: ($19) ReadDtcInformation - ReportMostRecentConfirmedDTC
Read -Mirror memory DTCs by status mask: ($19) ReadDtcInformation - Report mirror memory DTC by status mask
Read - Mirror memory extended record: ($19) ReadDtcInformation - Report mirror memory DTC extended data record by DTC number
Read - number of mirror memory DTCs: ($19) ReadDtcInformation - Report number of mirror memory DTC by status mask
Read - number emission DTCs: ($19) ReadDtcInformation - Report number of emission OBD DTC by status mask
Read - identified emission DTCs: ($19) ReadDtcInformation - Report emission OBD DTC by status mask
Read - fault detection counter: ($19) ReadDtcInformation - ReportDTCFaultDetectionCounter
Read - permanent DTCs: ($19) ReadDtcInformation - ReportDTCWithPermanentStatus
Clear DTC: ($14) ClearDiagnosticInformation
DtcStatusMask (vv)
DtcStatusAvailabilityMask (uu)
Format Identifier (zz)
DTC Table (ww)
StatusOfDtc (tt)
GroupOfDtc (yy)


MyECU
*Cdd_Demo
Edit DTCs
Navigation
DTC: B2600, 0x16 - Battery Voltage Low
Up
Down
Creation
B2600
0x18
New DTC
General
Authorization
Vehicle System Groups
Attributes
Aging Cycle Counter Threshold 0
Monitor Initialization Function not required
DTC Priority 1
DTC Severity noSeverity
Pre-Debounce Algorithm Counter based
Count-In step size 50
Count-out step size 10
Use Jump-Up (Prefailed) Yes
Hide empty lines
Show details
OK
Cancel
Help
修改优先级和滤波的步长
双击
DTC
Failure Type
Errortext
Failure type text
Errortext shortname
Set Condition
B2600 0x16 Battery Voltage Low Circuit Voltage Below Threshold
B2600 0x17 Battery Voltage High Circuit Voltage Above Threshold
Please click here to create a new element.
两个DTC一样的改法

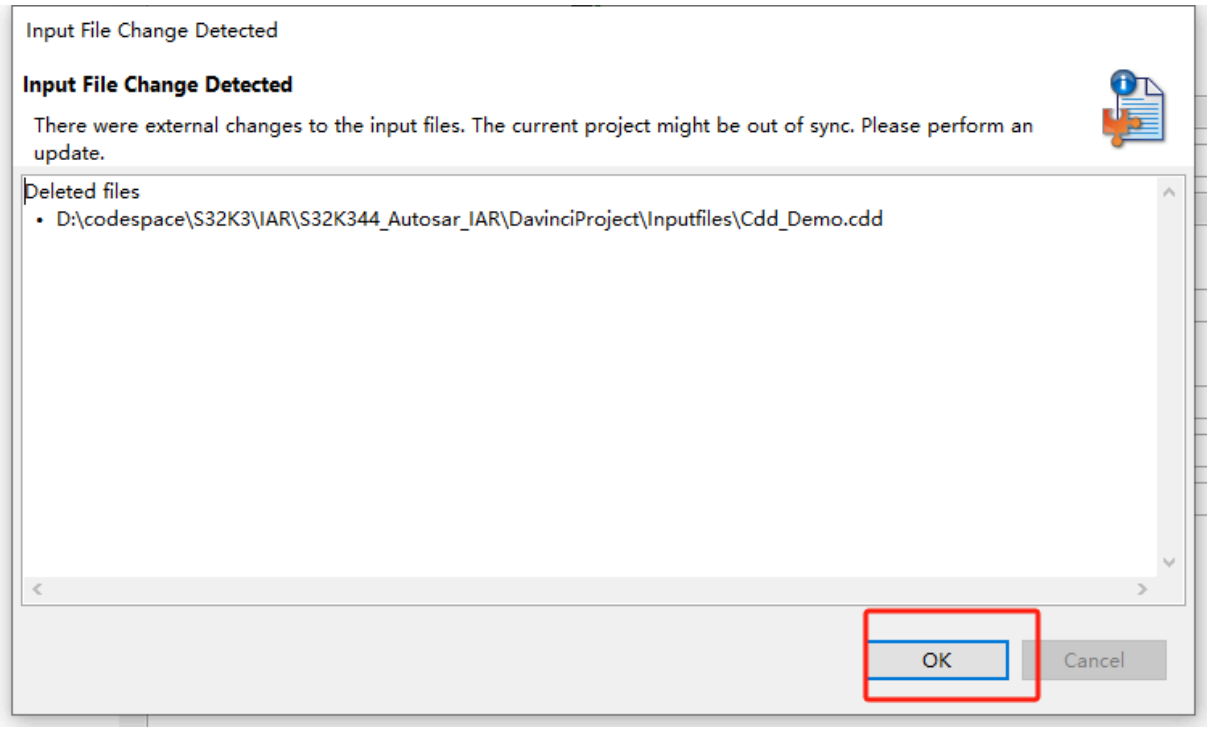


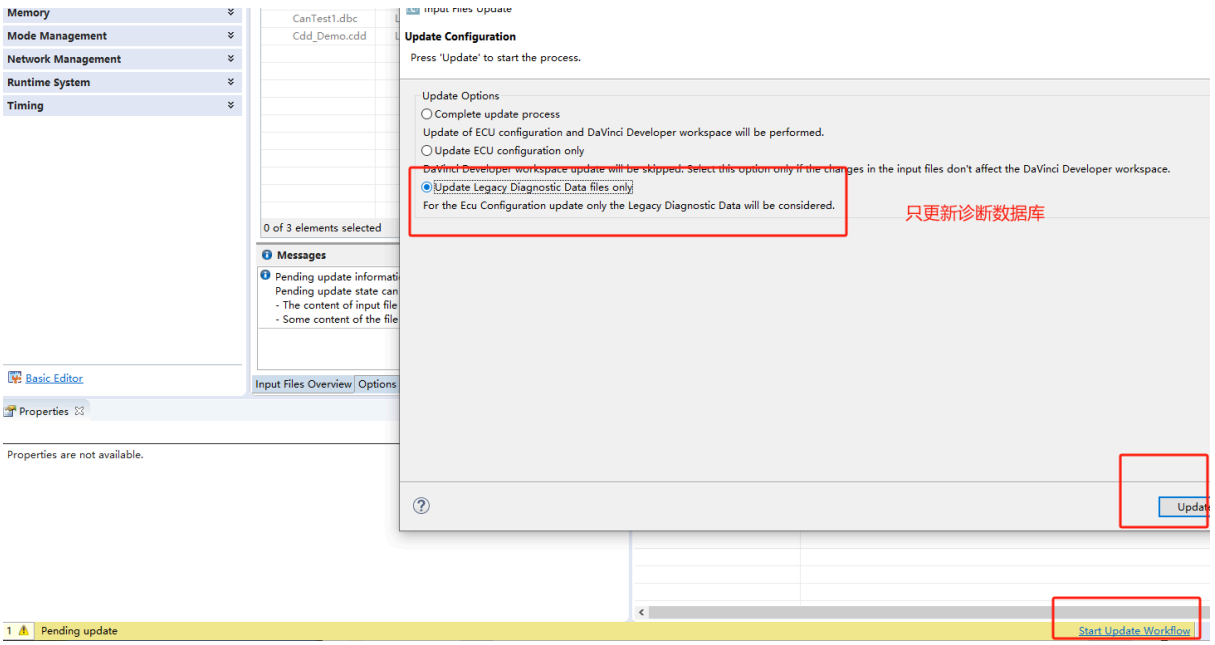


老化次数在davinci 的下面配置项中



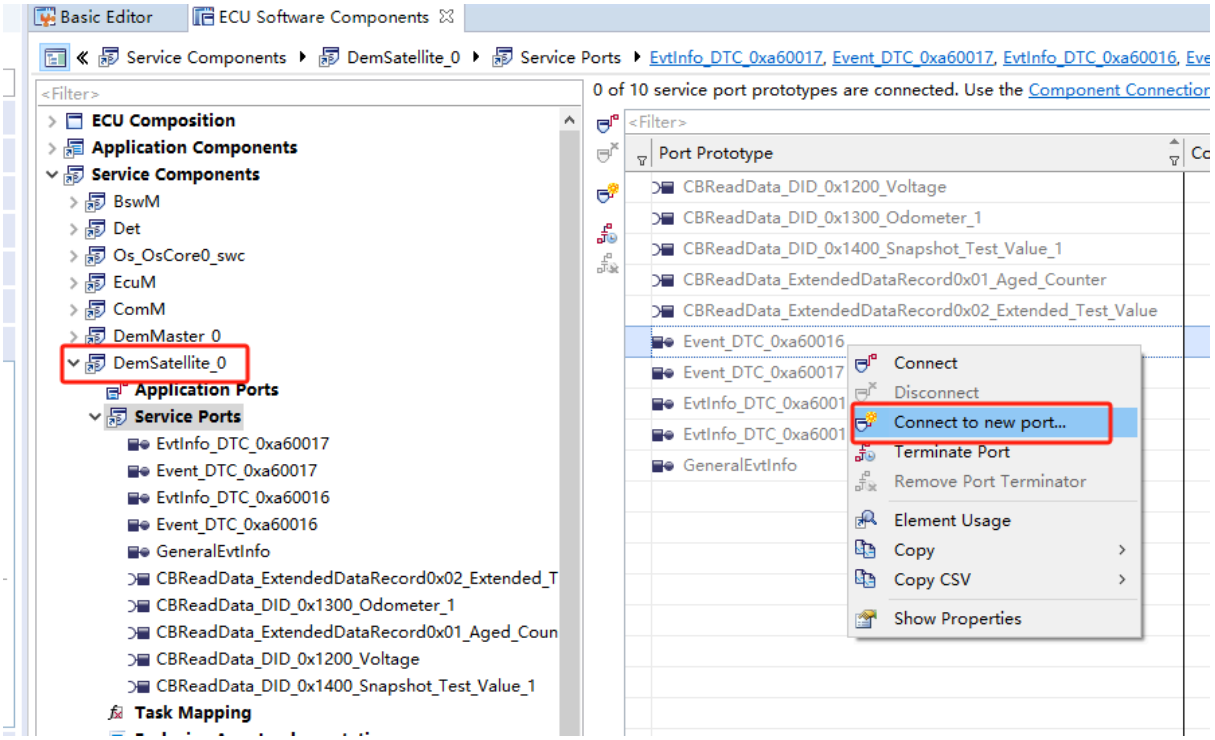
保存然后去davinci cfg中更新

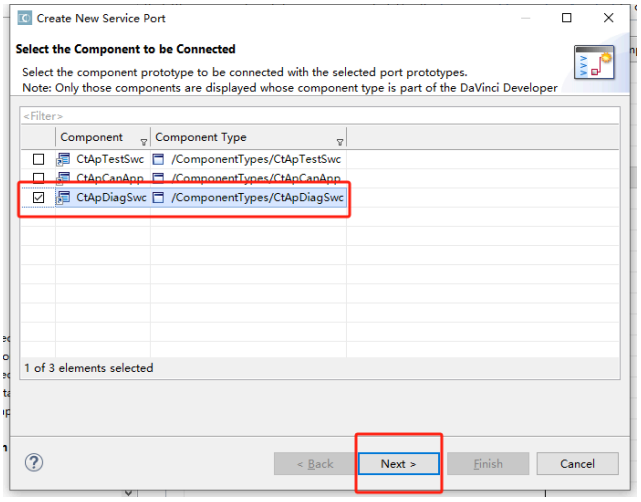




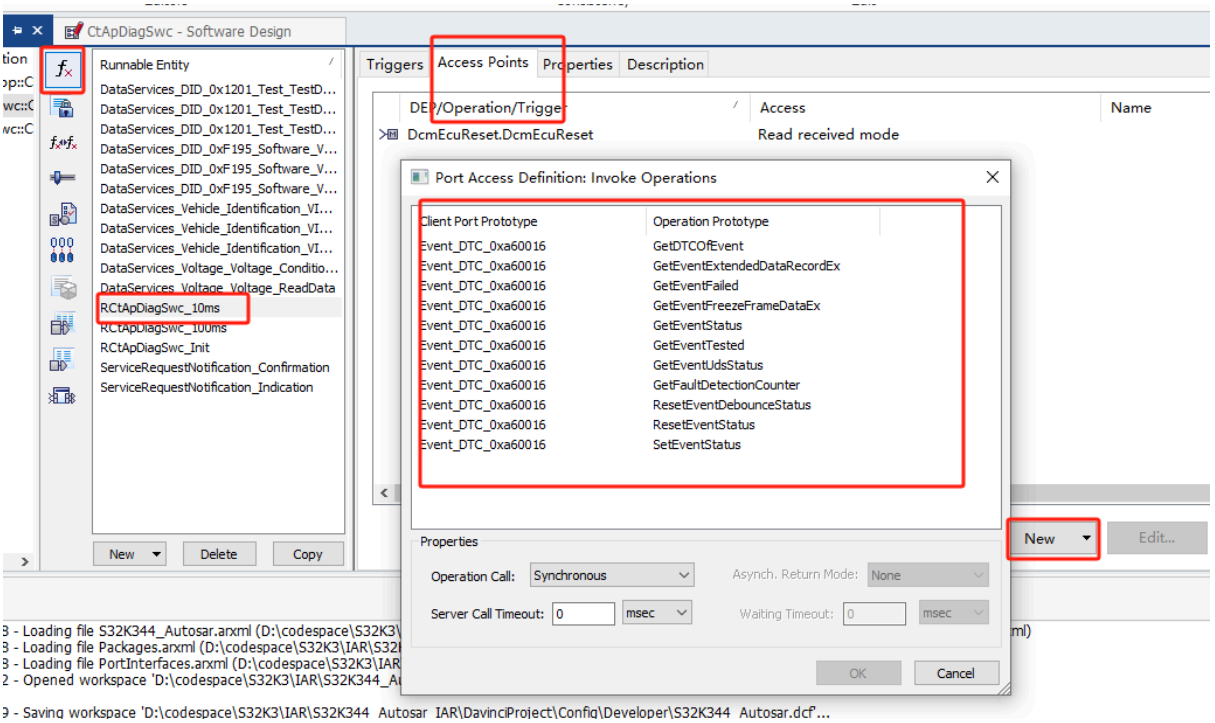
7.1.2、连接跟报DTC相关的接口

Event_DTC_0xa60016、Event_DTC_0xa60017、 OpCycle_PowerCycle一样的操作，右键新建port连接到CtApDiagSwc ，两个Event是用于设置报Dtc的，OpCycle_PowerCycle是操作循环，只有操作循环开始了，DTC才会开始记录

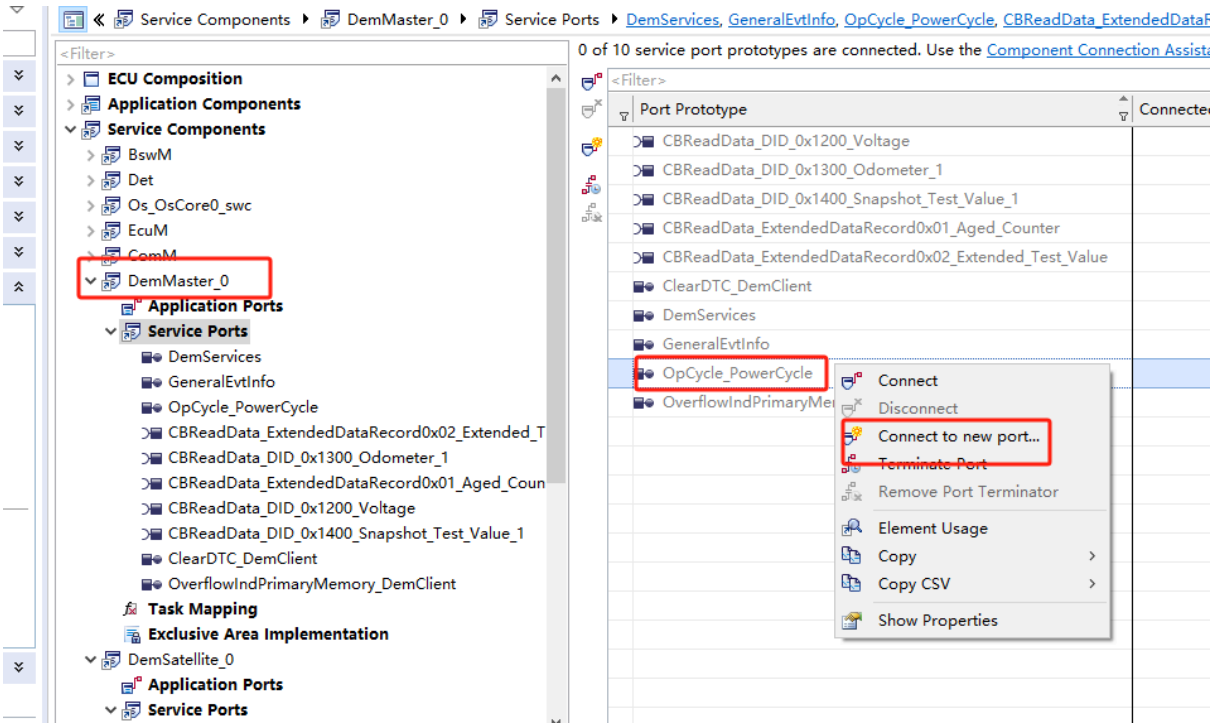


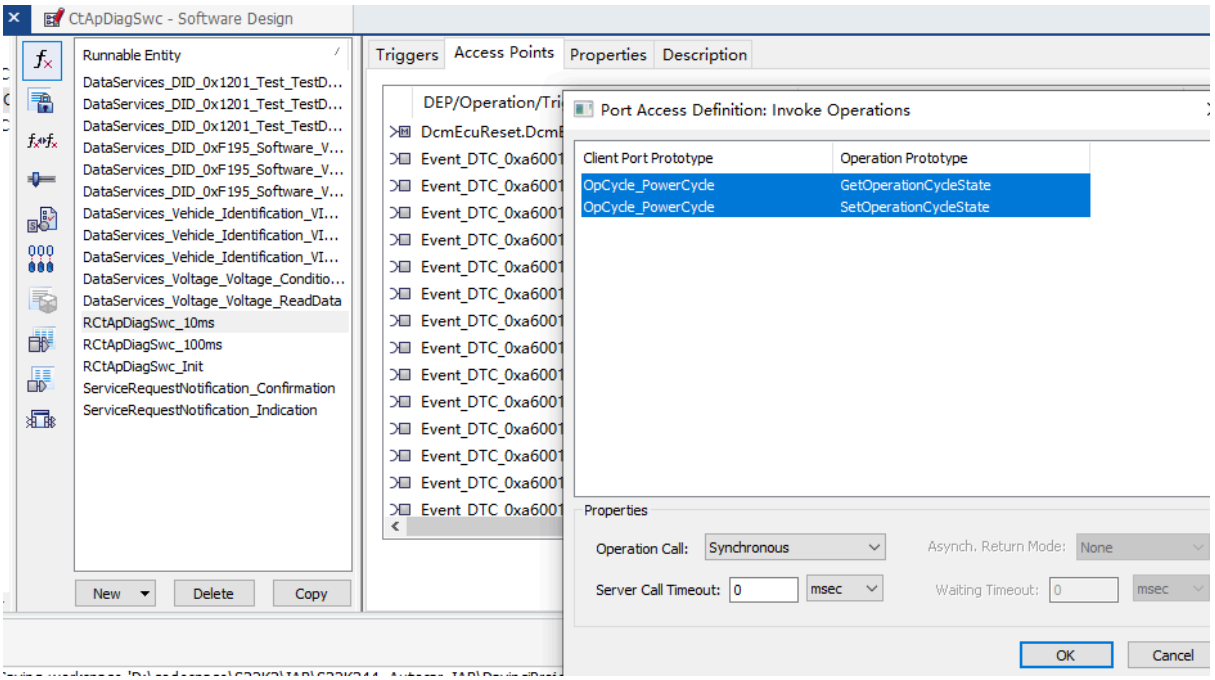


然后我们去developer invoke 接口即可



OpCycle_PowerCycle一样的操作





然后生成，Rte/os/Dcm/Dem， 最后生成Swc 代码即可

7.1.3、添加测试代码

Swc_init处添加 PowerCycle的操作循环开始， 这里SetEvent 用PREFAILED 和 PREPASSED, 代表是需要滤波

```
/**********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << End of documentation area >>                DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************/

FUNC(void, CtApDiagSwc_CODE) RCtApDiagSwc_Init(void) /* PRQA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
/**********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << Start of runnable implementation >>            DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApDiagSwc_Init
 **********************************************************************************************************/
  Rte_Call_OpCycle_PowerCycle_SetOperationCycleState(DEM_CYCLE_STATE_START);

/**********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << End of runnable implementation >>              DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************/
}

/**********************************************************************************************************
```

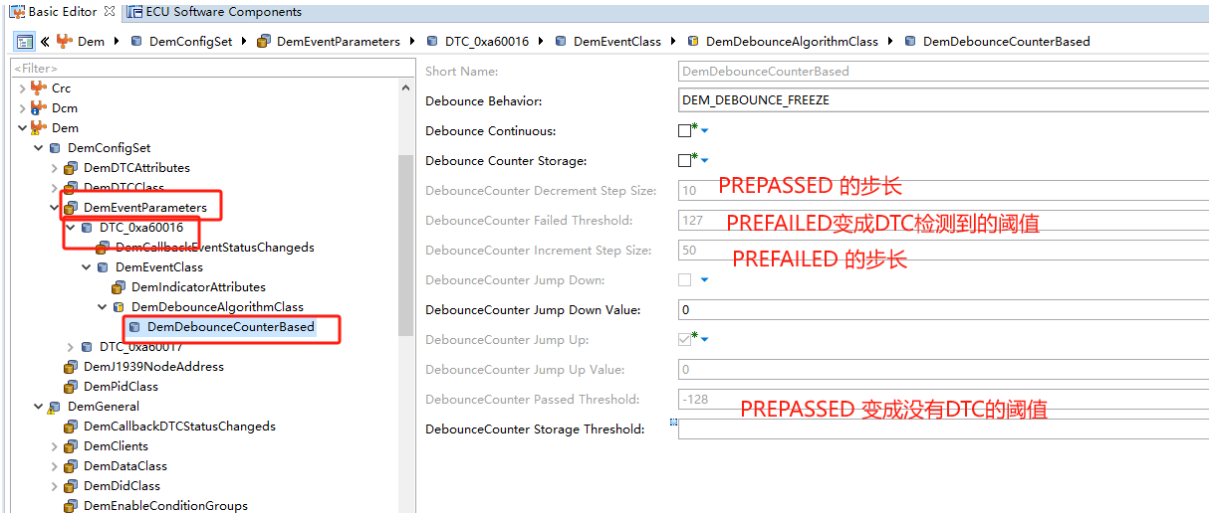
```
FUNC(void, CtApDiagSwc_CODE) RCtApDiagSwc_10ms(void) /* PRQA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
/**********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << Start of runnable implementation >>            DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApDiagSwc_10ms
 **********************************************************************************************************/
  uint8 Ret = Rte_Mode_DcmEcuReset_DcmEcuReset();
  if(Ret != RTE_MODE_DcmEcuReset_NONE){
    if((Ret == RTE_MODE_DcmEcuReset_JUMPTOBOOTLOADER) || (Ret == RTE_MODE_DcmEcuReset_EXECUTE) || (Ret == RTE_MODE_DcmEcuReset_HARD)){
      //Mcu_PerformReset();
    }
  }

  if(1==Dio_ReadChannel(DioConf_DioChannel_DioChannelPTB26_KEY1)){
    //Key1 Pressed
    Rte_Call_Event_DTC_0xa60016_SetEventStatus(DEM_EVENT_STATUS_PREFAILED);
  }else{
    Rte_Call_Event_DTC_0xa60016_SetEventStatus(DEM_EVENT_STATUS_PREPASSED);
  }
  if(1==Dio_ReadChannel(DioConf_DioChannel_DioChannelPTB19_KEY2)){
    //Key2 Pressed
    Rte_Call_Event_DTC_0xa60017_SetEventStatus(DEM_EVENT_STATUS_PREFAILED);
  }else{
    Rte_Call_Event_DTC_0xa60017_SetEventStatus(DEM_EVENT_STATUS_PREPASSED);
  }
```

Key1 按下时DTCA60016开始计数
否则减计数

Key2 按下时DTCA60017开始计数
否则减计数

滤波参数来自7.1配置的，在davinci如下



老化阈值的验证， 老化配置的是10， 当我们 有10次 操作循环后，历史DTC就会消除，测试代码参考如下：每10s 重新开启一次操作循环

```
FUNC(void, CtApDiagSwc_CODE) RCtApDiagSwc_100ms(void) /* PRQA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>             DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApDiagSwc_100ms
 *********************************************************************************************************************/
#if 0  //aging threthrod test
  static uint16 u16TempCnt = 0;
  if(u16TempCnt == 100){
    Rte_Call_OpCycle_PowerCycle_SetOperationCycleState(DEM_CYCLE_STATE_END);
  }
  else if(u16TempCnt == 150){
    Rte_Call_OpCycle_PowerCycle_SetOperationCycleState(DEM_CYCLE_STATE_START);
    u16TempCnt = 0;
  }
  u16TempCnt++;
#endif
/**********************************************************************************************************************
```

7.1.4、 报文 验证

01 验证

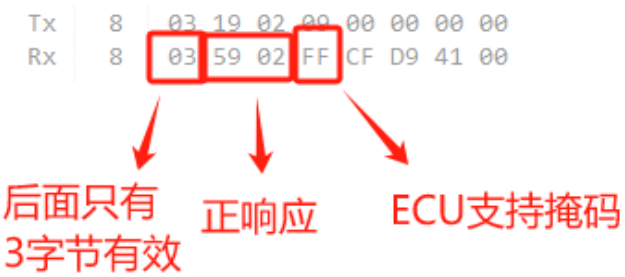
按键Key1/2都不按下，发送 19 01 09 （09代表查询当前DTC状态和历史DTC状态）

Key1/2都按下，发送19 01 09 , 可见 数量变为2了

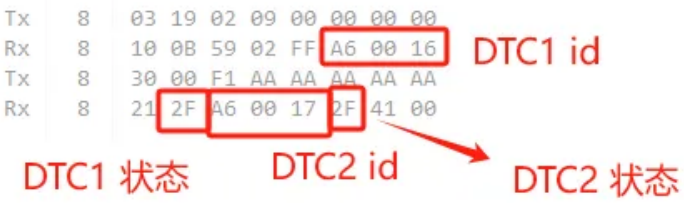
```
Tx   8   03 19 01 09 00 00 00 00
Rx   8   06 59 01 FF 01 00 02 00
```

02验证

按键Key1/2都不按下，发送 19 02 09 （09代表查询当前DTC状态和历史DTC状态）



Key1/2都有按下，发送19 02 09 ，回复了两个DTC id 及其状态



0A验证

0A 就是不管有没有错误，会把所有的DTC id及其当前状态返回

```
Tx   8   02 19 0A 00 00 00 00 00
Rx   8   10 0B 59 0A FF A6 00 16
Tx   8   30 00 F1 AA AA AA AA AA
Rx   8   21 20 A6 00 17 20 41 00
```

参考文档：

https://blog.csdn.net/galene/article/details/132537430

https://blog.csdn.net/Gagaaaaaa/article/details/135464587

https://blog.csdn.net/LLLJW_/article/details/145573048

https://blog.csdn.net/yessunday/article/details/132425899?spm=1001.2101.3001.6650.4&utm_medium=distribute.pc_relevant.none-task-blog-2%7Edefault%7EBlogCommendFromBaidu%7ERate-4-132425899-blog-104606208.235%5Ev43%5Epc_blog_bottom_relevance_base5&depth_1-utm_source=distribute.pc_relevant.none-task-blog-

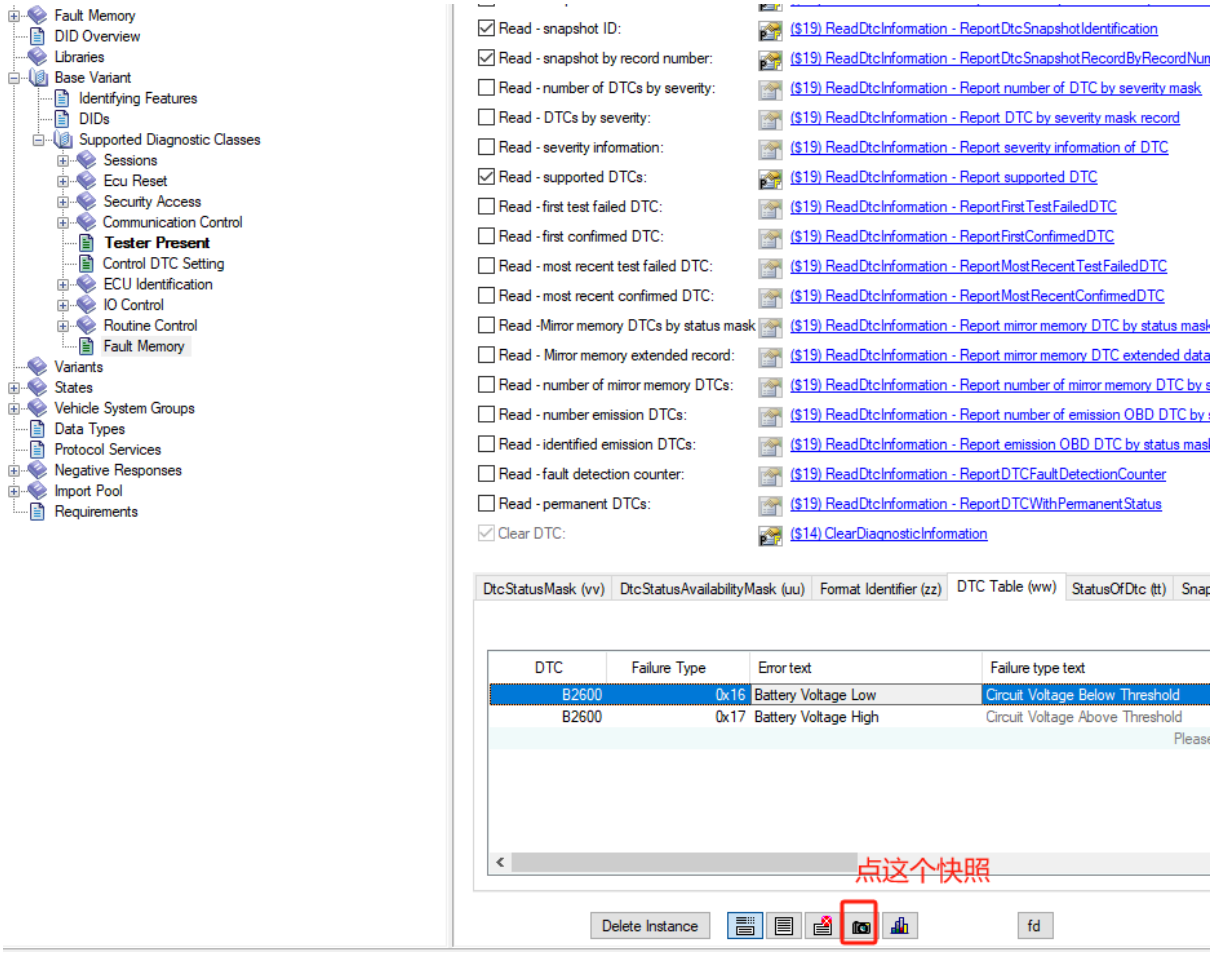
2%7Edefault%7EBlogCommendFromBaidu%7ERate-4-132425899-blog-104606208.235%5Ev43%5Epc_blog_bottom_relevance_base5&utm_relevant_index=9

https://blog.csdn.net/xyfx_fhw/article/details/104606208

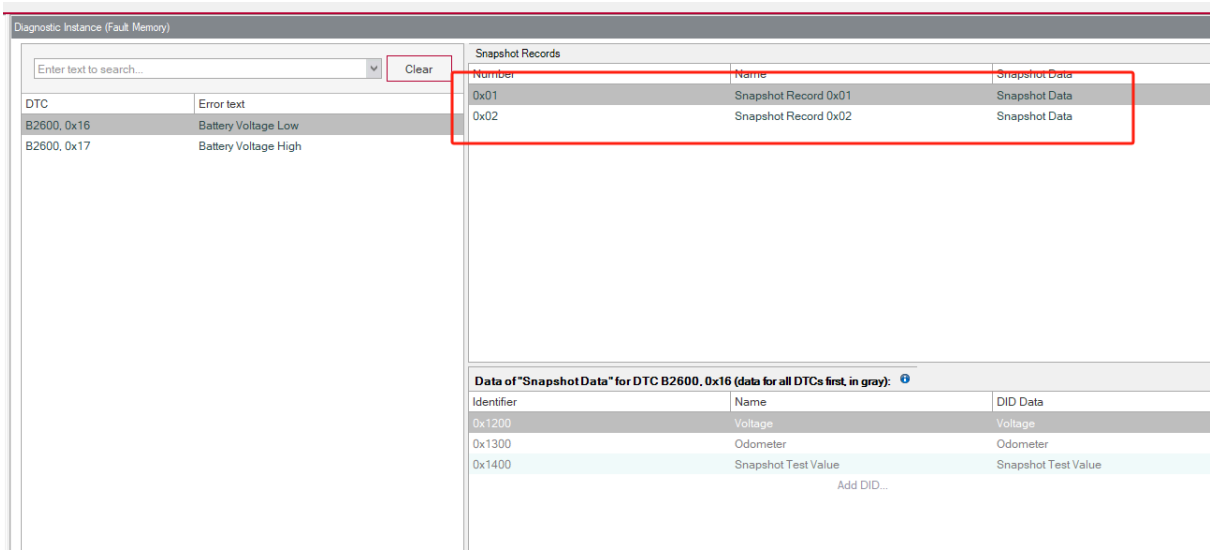
7.2、$19 03/04验证

这个sub 是快照

7.2.1、修改cdd

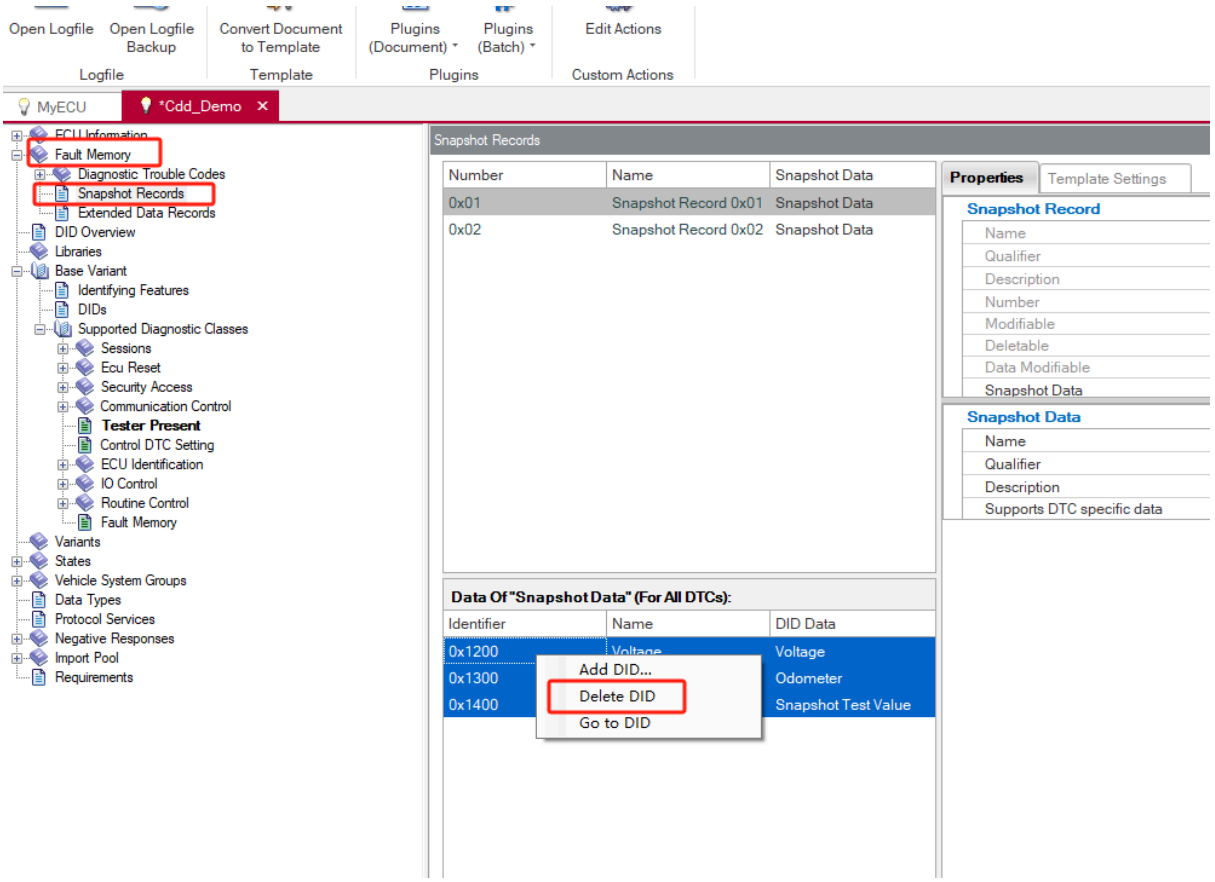


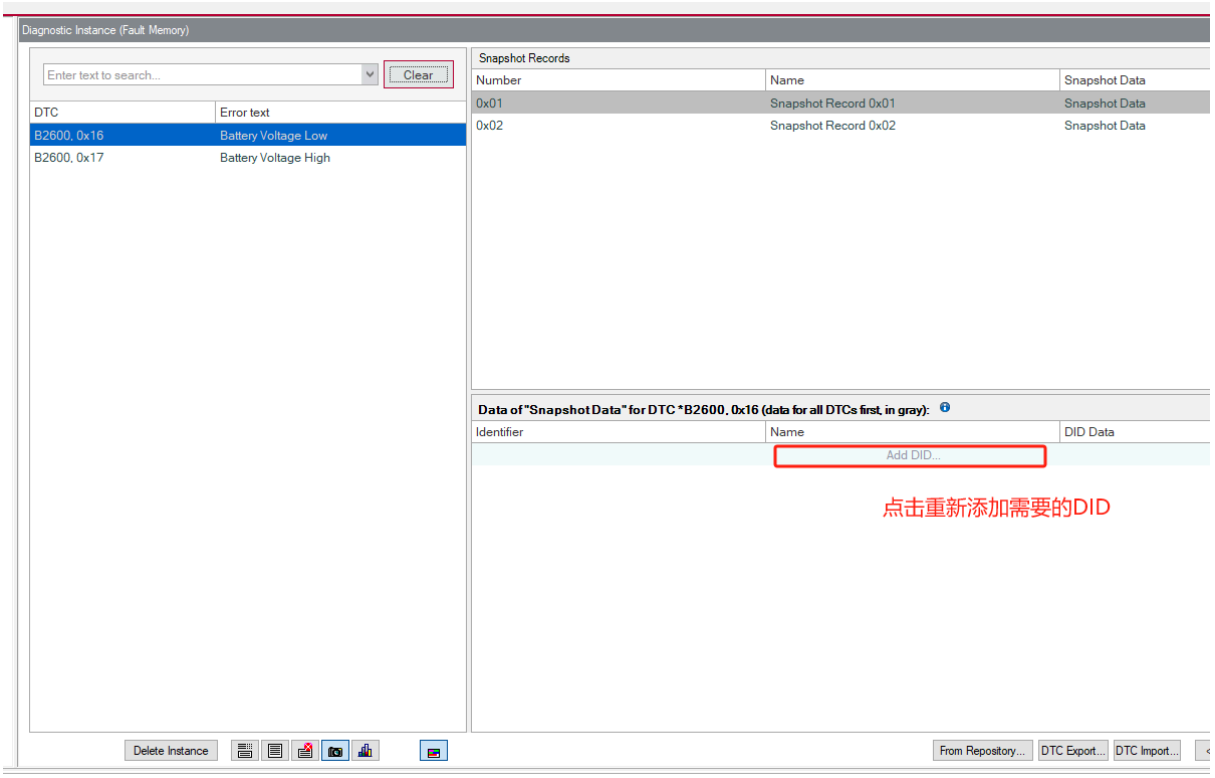
定义了两组快照，每个DTC都是，我们可以在下面给每个dtc 配置不一样的快照

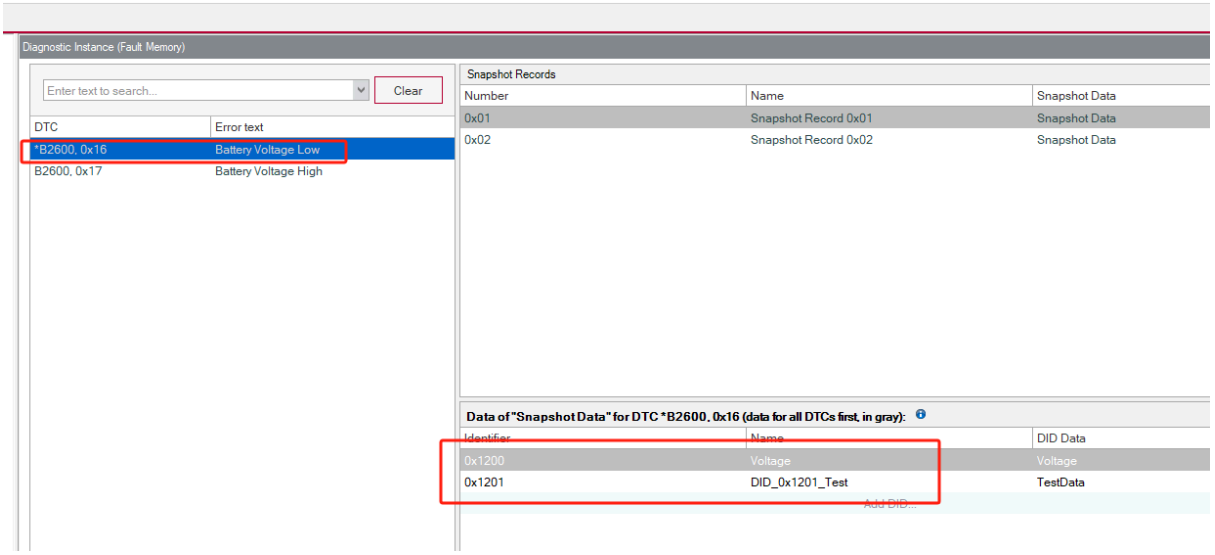


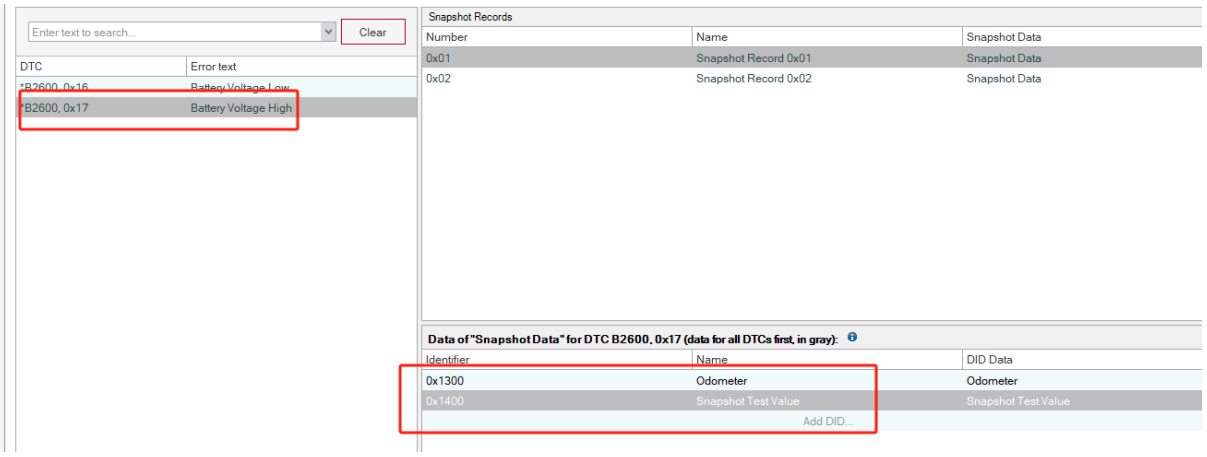
配置如下：

先删除， 这里的For All DTCs 适用所有dtc都有一套快照，我们不用这一种方式，我们使用每个DTC都是独立方式



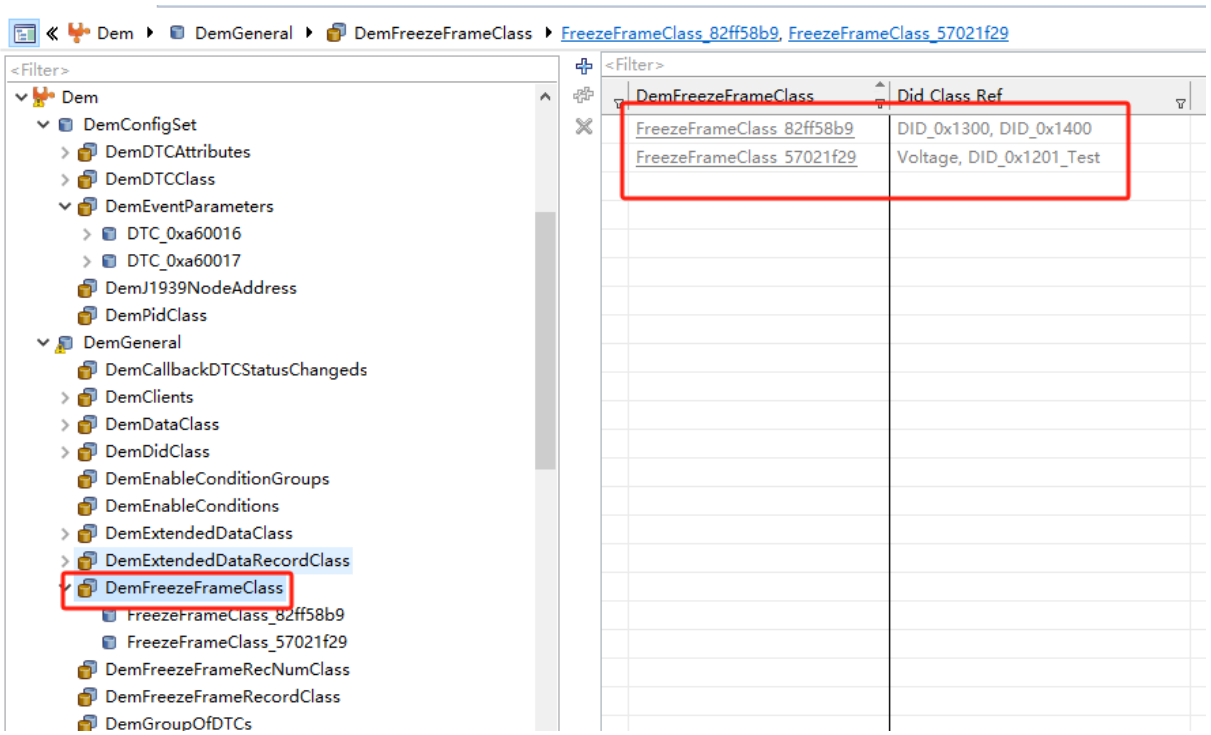

Diagnostic Instance (Fault Memory)
Enter text to search...
Clear
DTC
Error text
B2600, 0x16
Battery Voltage Low
B2600, 0x17
Battery Voltage High
Snapshot Records
Number
Name
Snapshot Data
0x01
Snapshot Record 0x01
Snapshot Data
0x02
Snapshot Record 0x02
Snapshot Data
Data of "Snapshot Data" for DTC *B2600, 0x16 (data for all DTCs first, in gray):
Identifier
Name
DID Data
Add DID...
点击重新添加需要的DID
Delete Instance
From Repository...
DTC Export...
DTC Import...


Diagnostic Instance (Fault Memory)
Enter text to search...
Clear
DTC
Error text
*B2600, 0x16
Battery Voltage Low
B2600, 0x17
Battery Voltage High
Snapshot Records
Number
Name
Snapshot Data
0x01
Snapshot Record 0x01
Snapshot Data
0x02
Snapshot Record 0x02
Snapshot Data
Data of "Snapshot Data" for DTC *B2600, 0x16 (data for all DTCs first, in gray):
Identifier
Name
DID Data
0x1200
Voltage
Voltage
0x1201
DID_0x1201_Test
TestData
Add DID...

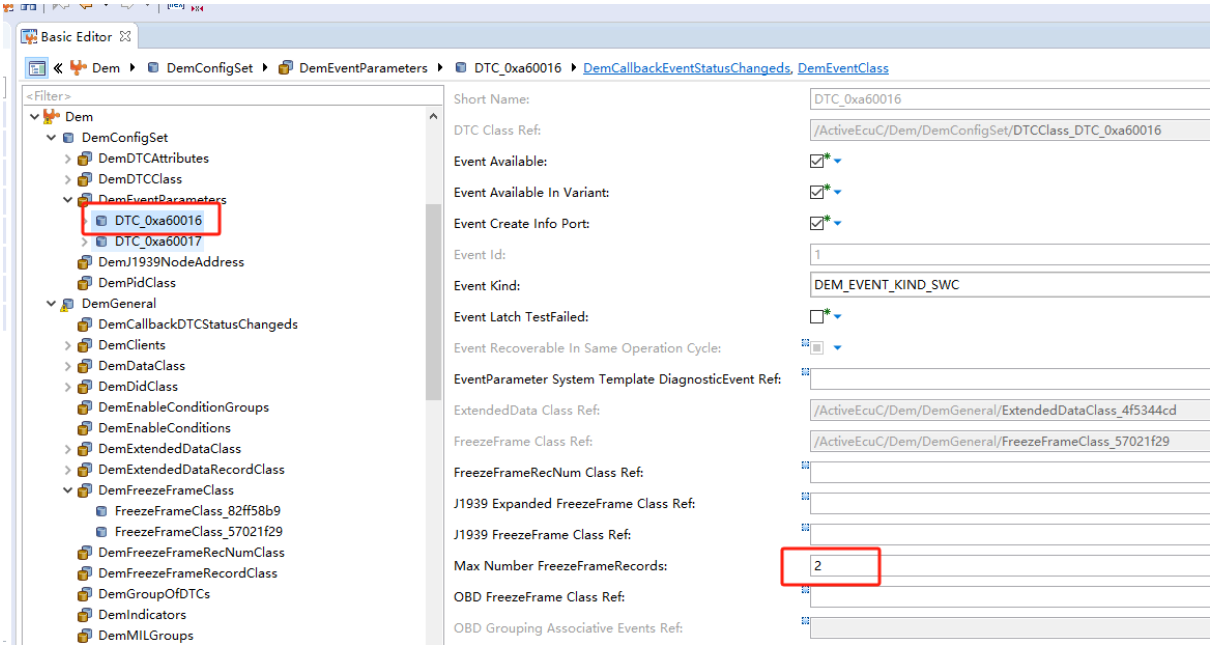


保存同步到davinci 更新

可见冻结帧这里有两个不一样的配置，对应我们两个DTC， 这里代表发生DTC 时记录的DID数据，就是快照

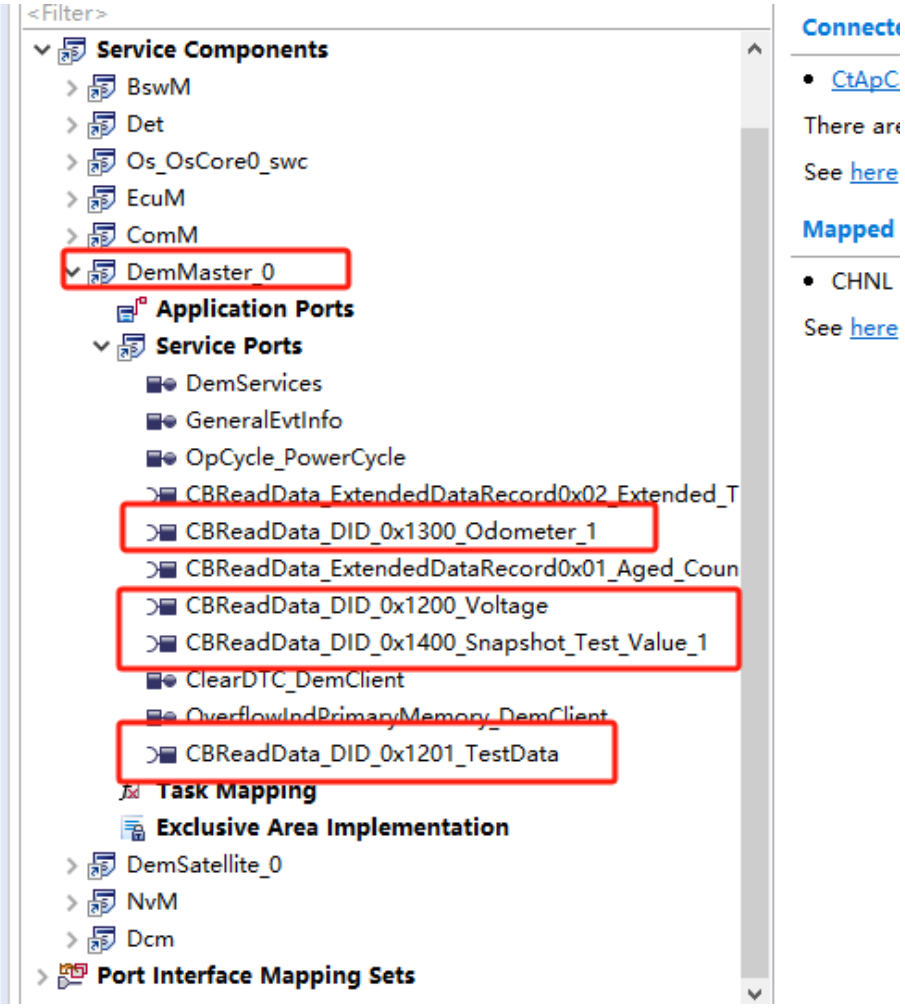


快照组数量可以在下面修改



7.2.2、连接跟报DTC相关的接口

把DemMaster_0的如下四个CBReadData 按照7.1.2 的步骤新建port 连接到CtApDiagSwc



生成Dcm/Dem/Os/Rte，然后再生成Swc 代码

7.2.3、添加测试代码

```
FUNC(Std_ReturnType, CtApDiagSwc_CODE) CBReadData_DID_0x1200_Voltage_ReadData(P2VAR(DataPrimitiveType_uint16, AUTOMATIC, RTE_CTAPDIAGSWC_APPL_VAR) Data
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>              DO NOT CHANGE THIS COMMENT!
 * Symbol: CBReadData_DID_0x1200_Voltage_ReadData (returns application error)
 **********************************************************************************************************************/
  *Data = 0x3344;
  return RTE_E_OK;

/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << End of runnable implementation >>                DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/
}
```

```
FUNC(Std_ReturnType, CtApDiagSwc_CODE) CBReadData_DID_0x1201_TestData_ReadData(P2VAR(DataPrimitiveType_uint8, AUTOMATIC, RTE_CTAPDIAGSWC_APPL_VAR) Data)
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>              DO NOT CHANGE THIS COMMENT!
 * Symbol: CBReadData_DID_0x1201_TestData_ReadData (returns application error)
 **********************************************************************************************************************/
  static uint8 TestData = 0x10;
  Data[0] = TestData++;
  return RTE_E_OK;

/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << End of runnable implementation >>                DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/
}
```

```
FUNC(Std_ReturnType, CtApDiagSwc_CODE) CBReadData_DID_0x1300_Odometer_1_ReadData(P2VAR(uint8, AUTOMATIC, RTE_CTAPDIAGSWC_APPL_VAR) Data
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>             DO NOT CHANGE THIS COMMENT!
 * Symbol: CBReadData_DID_0x1300_Odometer_1_ReadData (returns application error)
 **********************************************************************************************************************/
  Data[0] = 0x66;
  Data[1] = 0x77;
  Data[2] = 0x88;
  return RTE_E_OK;

/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << End of runnable implementation >>               DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/
}
```

```
FUNC(Std_ReturnType, CtApDiagSwc_CODE) CBReadData_DID_0x1400_Snapshot_Test_Value_1_ReadData(P2VAR(DataPrimitiveType_uint16, AUTOMATIC, RTE_CTAPDIAGSWC_APPL_VAR) Data)
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>             DO NOT CHANGE THIS COMMENT!
 * Symbol: CBReadData_DID_0x1400_Snapshot_Test_Value_1_ReadData (returns application error)
 **********************************************************************************************************************/
  *Data = 0x7788;
  return RTE_E_OK;

/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << End of runnable implementation >>               DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/
}
```

7.2.4、报文验证

03 验证，（把产生过DTC的快照都列出来， DTC id + 快照组号 依次排列）

产生一次DTC保存后

```
Tx   8   02 19 03 00 00 00 00 00
Rx   8   06 59 03 A6 00 16 01 00
```

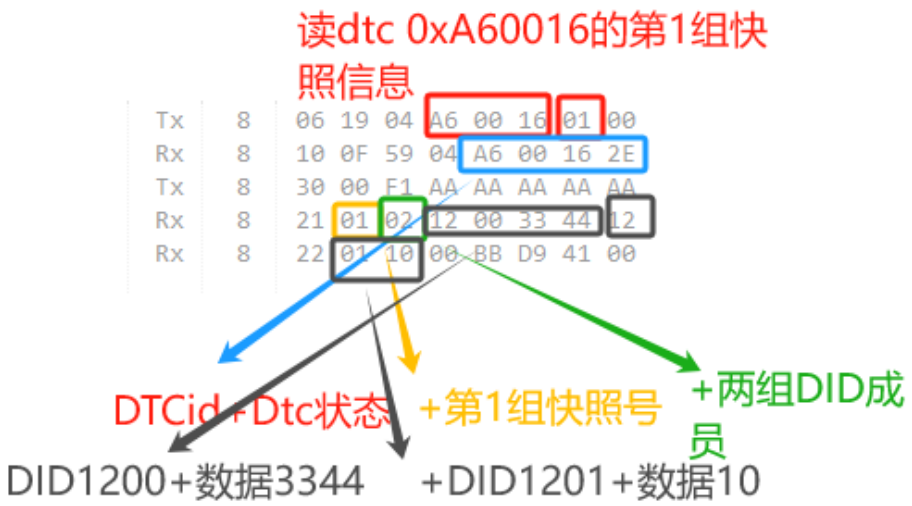
DTC id

发送的快照数，最大不超过配置的数量

产生多次DTC后，最多两组

```
Tx   8   02 19 03 00 00 00 00 00
Rx   8   10 0A 59 03 A6 00 16 01
Tx   8   30 00 F1 AA AA AA AA AA
Rx   8   21 A6 00 16 02 D9 41 00
```

04验证，读取某个DTC ，某组快照信息



回复数据排列 ： dtcid + dtc status + 快照组号 + 快照包含的DID数量 + [DID1 id + 数据] +. ..... + [DIDx id + 数据]

参考： https://blog.csdn.net/LOVE135149/article/details/123074190

经过初步验证，当快照组号用满之后，都没再发生DTC，也只会一直记录在最后一组

7.3、$19 06验证

DTC Extended Data：扩展数据就是存储故障的扩展信息，说了跟没说一样。但就是这么个意思。常见的有：故障发生计数器，等待故障计数器，老化计数器，已老化计数器，故障指示灯计数器

a）Fault Occurrence Counter(故障发生计数器)，就是记录故障发生的次数，也就是说某个故障DTC Status的bit0只要bit0发生一次置1，这个计数器就加1

b）Fault Pending Counter(等待故障计数器)，就是pending DTC发生的次数，对应DTC Status的bit2置为1

c）Aging Counter(老化计数器)，表示正在老化的计数，假设老化周期为40，则每老化一次Aging Counter加1，最大值为40，达到40后又从0开始

d）Aged Counter(已老化计数器)，已完成老化的计数，在Aging Counter达到40时，Aged Counter才会加1，最大值为0XFF

e）Malfunction Indicator counter(故障指示灯计数器)，用于传输故障激活时OBD系统已经运行的时间（发动机的工作时间）

执行了ClearDiag操作之后，aging counter和aged counter会同时Reset清零。

请求：

[SID] + [sub-function] + [要读取的DTC code] + [扩展数据编号]

Table 259 — Request message definition - sub-function = reportDTCExtDataRecordByDTCNumber, reportMirrorMemoryDTCExtDataRecordByDTCNumber

A_Data byte	Parameter Name	Cvt	Byte Value	Mnemonic
#1	ReadDTCInformation Request SID	M	0x19	RDTCI
#2	sub-function = [reportType = reportDTCExtDataRecordByDTCNumber reportMirrorMemoryDTCExtDataRecordByDTCNumber]	M	 0x06 0x10	LEV_ RDTCEDRBDN RMDEDRBDN
 #3 #4 #5	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte]	 M M M	 0x00 – 0xFF 0x00 – 0xFF 0x00 – 0xFF	DTCMREC_ DTCHB DTCMB DTCLB
#6	DTCExtDataRecordNumber	M	0x00 – 0xFD	DTCEDRN

www.cartech8.com

回复：

[SID + 0x40] + [sub-function] + [DTC code + DTC Status] + [扩展数据编号] + [扩展数据]

Table 276 — Response message definition - sub-function = reportDTCExtDataRecordByDTCNumber and reportMirrorMemoryDTCExtDataRecordByDTCNumber

A_Data byte	Parameter Name	Cvt	Byte Value	Mnemonic
#1	ReadDTCInformation Response SID	M	0x59	RDTCIPR
#2	reportType = [reportDTCExtDataRecordByDTCNumber reportMirrorMemoryDTCExtDataRecordByDTCNumber]	M	 0x06 0x10	LEV_ RDTCEDRBD RMDEDRBDN
 #3 #4 #5 #6	DTCAndStatusRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 0x00 – 0xFF 0x00 – 0xFF 0x00 – 0xFF 0x00 – 0xFF	DTCASR_ DTCHB DTCMB DTCLB SODTC
#7	DTCExtDataRecordNumber#1	C_1	0x00-0xFD	DTCEDRN
 #8 : #8+(p-1)	DTCExtDataRecord[]#1 = [extendedData#1 byte#1 : extendedData#1 byte#p]	 C_1 C_1 C_1	 0x00 – 0xFF : 0x00 – 0xFF	DTCSSR_ EDD11 : EDD1p
:	:	:	:	:
#t	DTCExtDataRecordNumber#x	C_2	0x00 – 0xFD	DTCEDRN
 #t+1 : #t+1+(q-1)	DTCExtDataRecord[]#x = [extendedData#x byte#1 : extendedData#x byte#q]	 C_2 C_2 C_2	 0x00 – 0xFF : 0x00 – 0xFF	DTCSSR_ EDDx1 : EDDxq

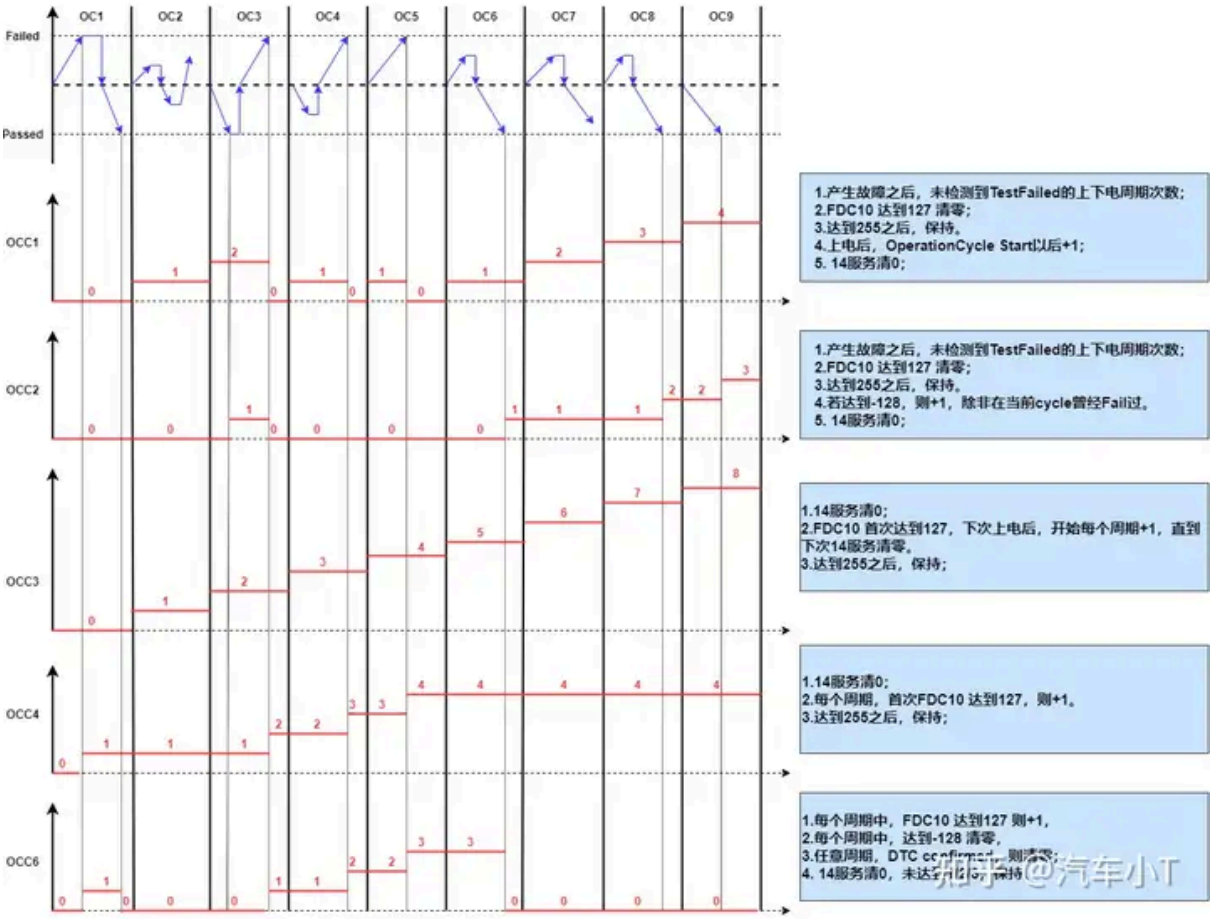
C_1: The DTCExtDataRecordNumber and the extendedData in the DTCExtDataRecord parameter are only present if at least one DTCExtDataRecord is available to be reported.

C_2: The DTCExtDataRecordNumber and the extendedData in the DTCExtDataRecord parameter are only present if all records are requested to be reported (DTCExtDataRecordNumber set to 0xFE or 0xFF in the request message) and at least one record is available to be reported.

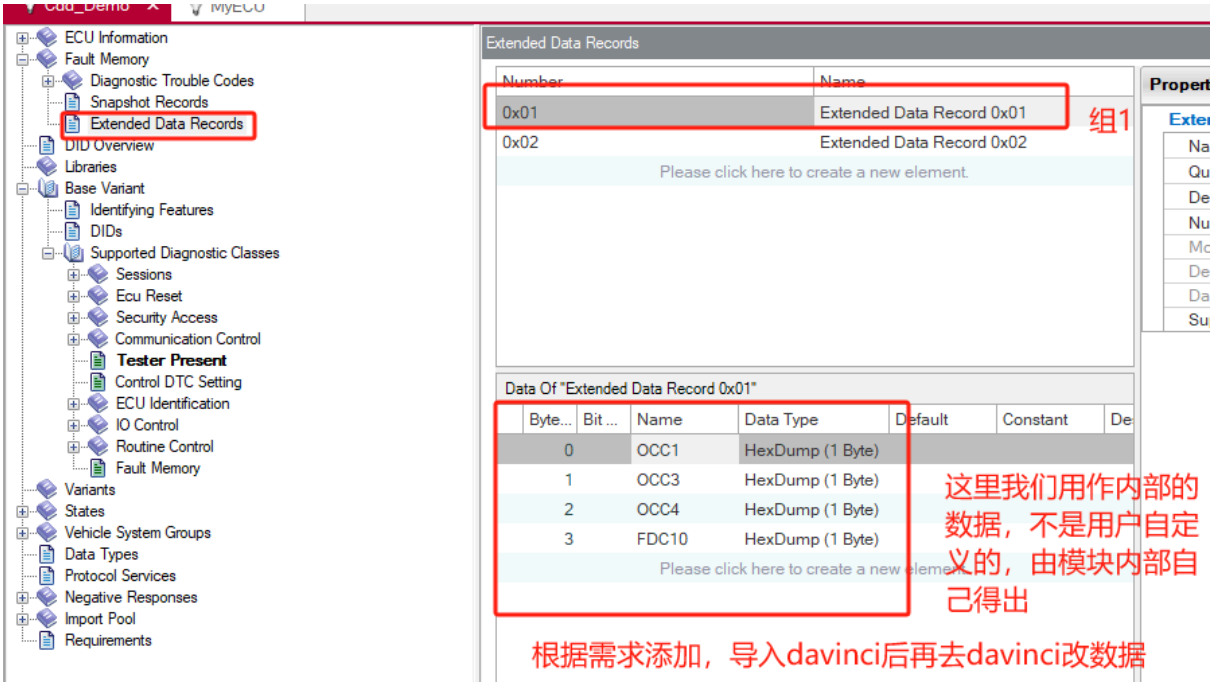
www.cartech8.com

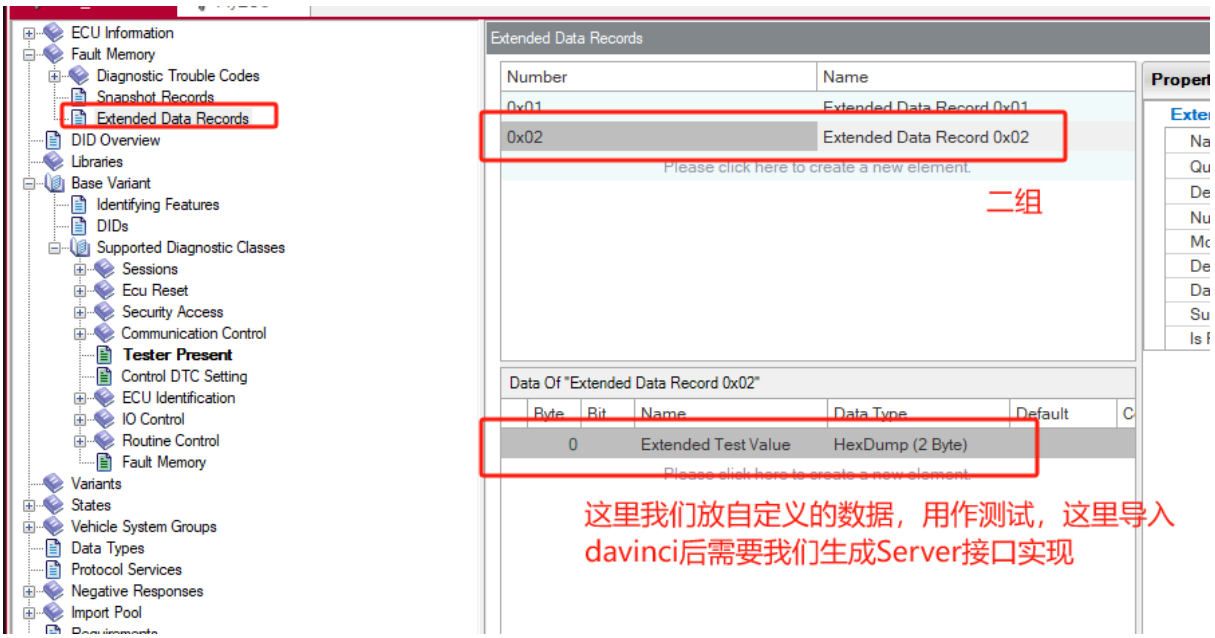
https://www.zhihu.com/question/519405504

> WWH-OBD　World Wide Harmonized On-Board-Diagnostic
> OC　Occurrence Count　发生计数器
> SW-C　Software Component　软件组件
> NvM　Non-Volatile Random Access Memory Manager　非易失性随机存取存储器管理
> OCC1　自上一个 FDC 达到 127 的操作循环后经过的操作循环个数。
> OCC2　自上一个 FDC 达到 127 的操作循环后经过的该故障测试通过的操作循环个数。
> OCC3　自第一次 FDC 达到 127 的操作循环后经过的操作循环的个数。
> OCC4　FDC 达到 127 的操作循环的个数。
> OCC5　自上一次故障清除后 MIL 熄灭后经历的暖机循环的个数。
> OCC6　连续的 FDC 达到 127 的操作循环的个数。
> MID　Monitor ID　诊断监视器标识符。
> TID　Test ID　测试标识符。
> UaSID　Units and Scaling ID　单位和缩放比例标识符。
> Dcm　Electronic Control Unit+B438:C463　诊断管理模块
> DSL　Diagnostic Session Layer.　诊断链路层
> DSD　Diagnostic Service Dispatcher　诊断服务分发层

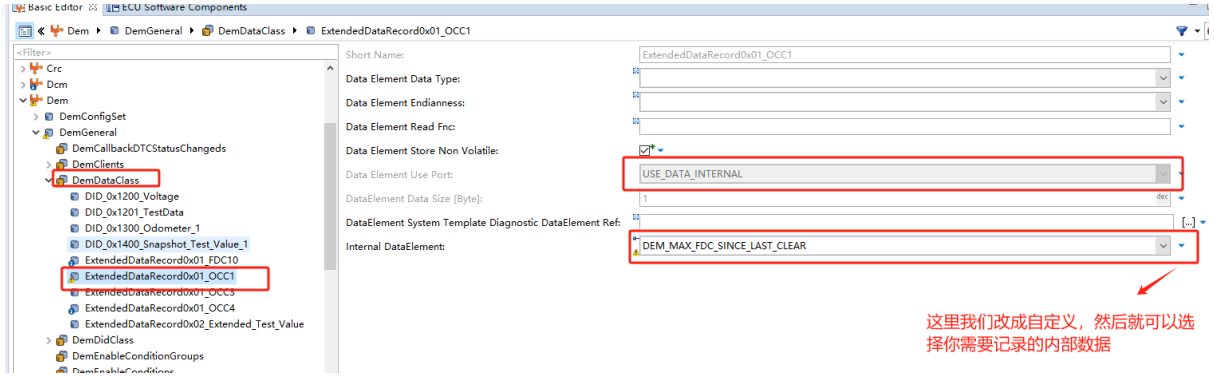


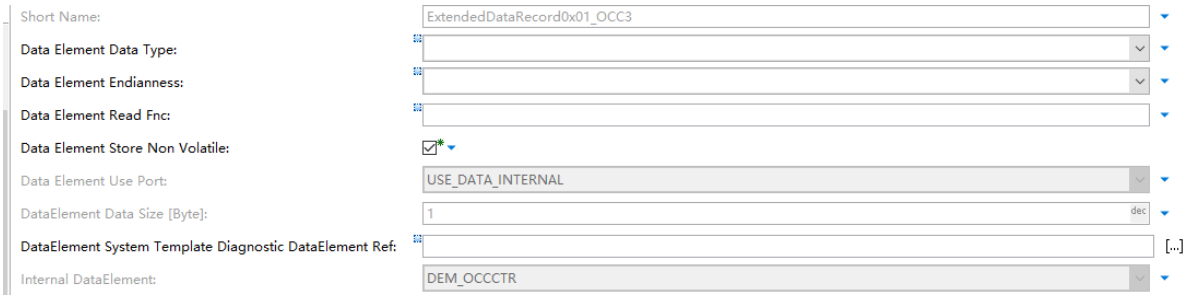
7.3.1、修改cdd

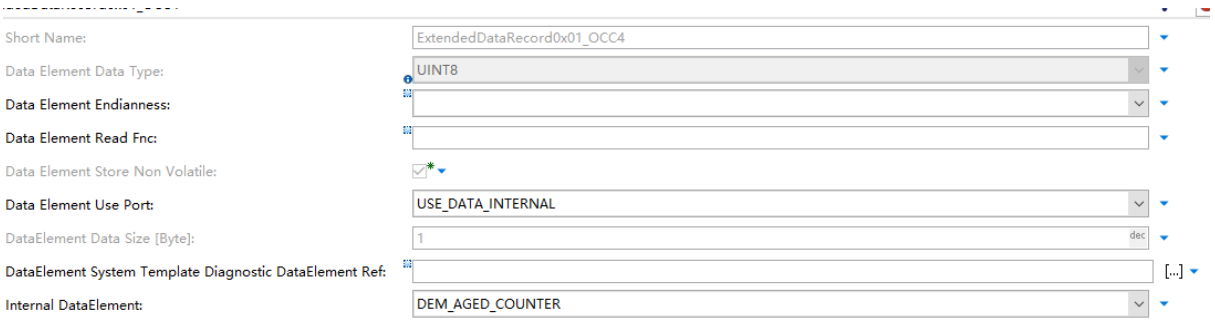


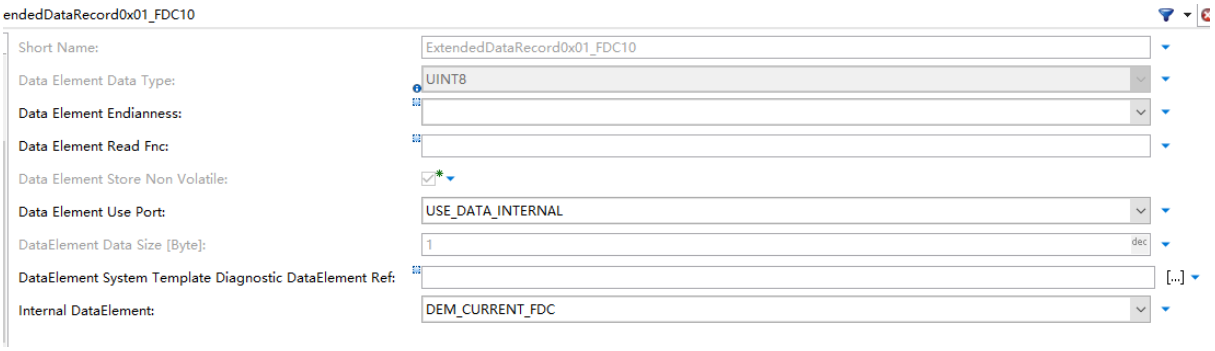


7.3.2、连接用户ext的port和修改ext为内部数据





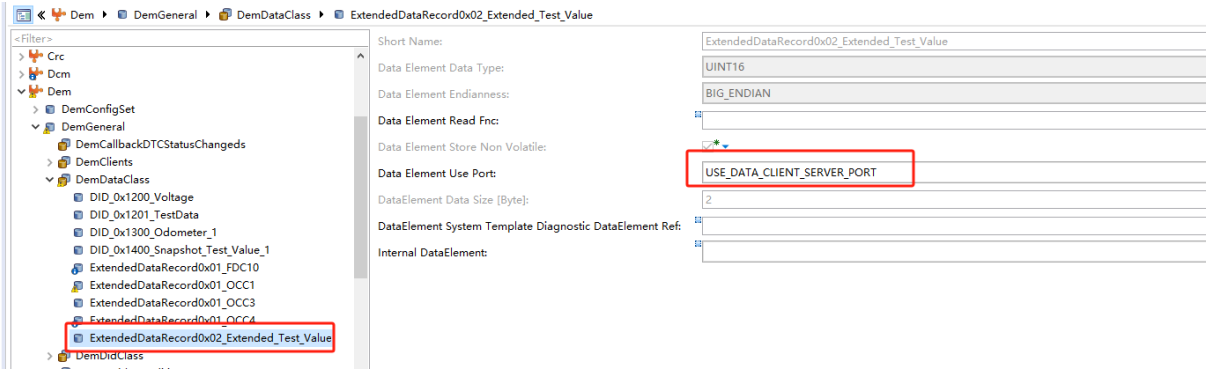


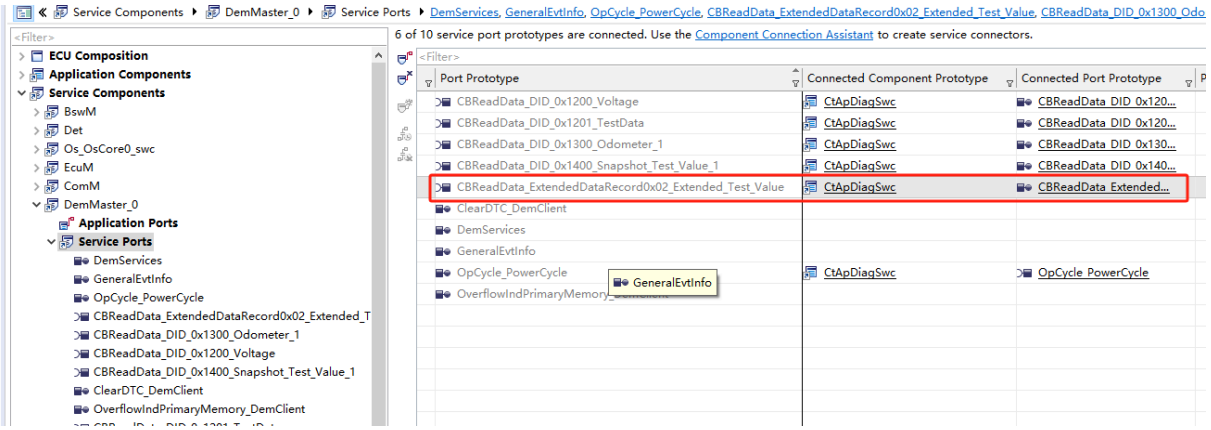


上面这些数据不用我们新建port去实现，Dem模块内部会得出数据

然后用户自定义的数据如下：

配置为USE_DATA_CLIENT_SERVER_PORT，我们需要去DemMaster去连接到CtApDiagSwc





代码

```
FUNC(Std_ReturnType, CtApDiagSwc_CODE) CBReadData_ExtendedDataRecord0x02_Extended_Test_Value_ReadData P2VAR(DataPrimitiveType_uint16, AUTOMATIC, RTE_CTAPDIAGSWC_APPL_VAR) Data)
{
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>            DO NOT CHANGE THIS COMMENT!
 * Symbol: CBReadData_ExtendedDataRecord0x02_Extended_Test_Value_ReadData (returns application error)
 *********************************************************************************************************/
  static uint16 ExtData2 = 0x1001;
  *Data = ExtData2++;
  return RTE_E_OK;
```

7.3.3、报文验证

查 DTC A60016的扩展数据1

```
Tx   8   06 19 06 A6 00 16 01 00
Rx   8   10 0B 59 06 A6 00 16 2E
Tx   8   30 00 F1 AA AA AA AA AA
Rx   8   21 01 7F 01 00 80 41 00
```

查DTC A60017的扩展数据2

```
Tx   8   06 19 06 A6 00 16 02 00
Rx   8   10 09 59 06 A6 00 16 2E
Tx   8   30 00 F1 AA AA AA AA AA
Rx   8   21 02 10 01 AF D9 41 00
```

7.4、$14服务

清除DTC

请求：[SID] + [要清除的DTC code，三字节]　　(FFFFFF 代表清除所有DTC)

A_Data 字节	参数名称	Cvt	字节值	助记符
#1	ClearDiagnosticInformation（清除诊断信息）的请求服务标识符	M	0x14	CDTCI
#2 #3 #4	groupOfD TC[] = [groupOfDTCHighByte groupOfDTCMiddleByte groupOfDTCLowByte]	M M M	0x00 – 0xFF 0x00 – 0xFF 0x00 – 0xFF	GODTC_ HB MB LB

7.5、$85服务

该服务用于停止或继续ECU中 DTC 状态位的更新（是否记录DTC）

DTC控制服务的请求报文的数据格式也很简单，[SID] + [sub-function]

子功能为DTCSettingType，常用的就两个，0X01和0X02，其它的要不是保留，要不是汽车制造商或供应商自定义的。

0X01：表示DTC设置类型为on，服务端收到请求后恢复DTC状态的更新

0X02：表示DTC设置类型为off，服务端收到请求后停止DTC状态的更新

发送 85 01 或者85 02